Ninja vs Buzo

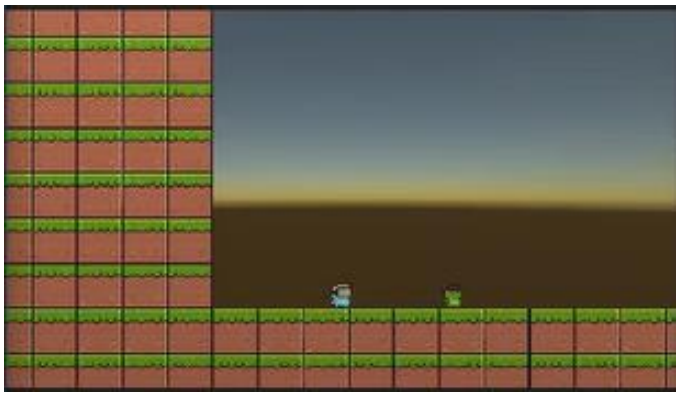
Juego multijugador

1. Introducción 2
2. Creación de escenarios 2
   a. Escena Cortinilla 2
   b. Escena GameScene 6
4. Personaje Ninja Frog y sus animaciones 10
   5. Scripts del movimiento del personaje 17
6. Personaje pequeño Buzo y sus animaciones 20
7. Configuración Escena Multijugador 24
8. Cambio de escena 27
9. Propiedad (owner) 29
10. Sincronización Cámara 29
11. RPC flip usuario 31

1. Introducción

Se pretende realizar un juego multijugador con 2 usuarios. Cada uno de los jugadores tendrá el control de su propio personaje y con su cámara. A cada uno de estos dos jugadores tendrá asignada una cámara que a perseguir a cada uno de los jugadores.

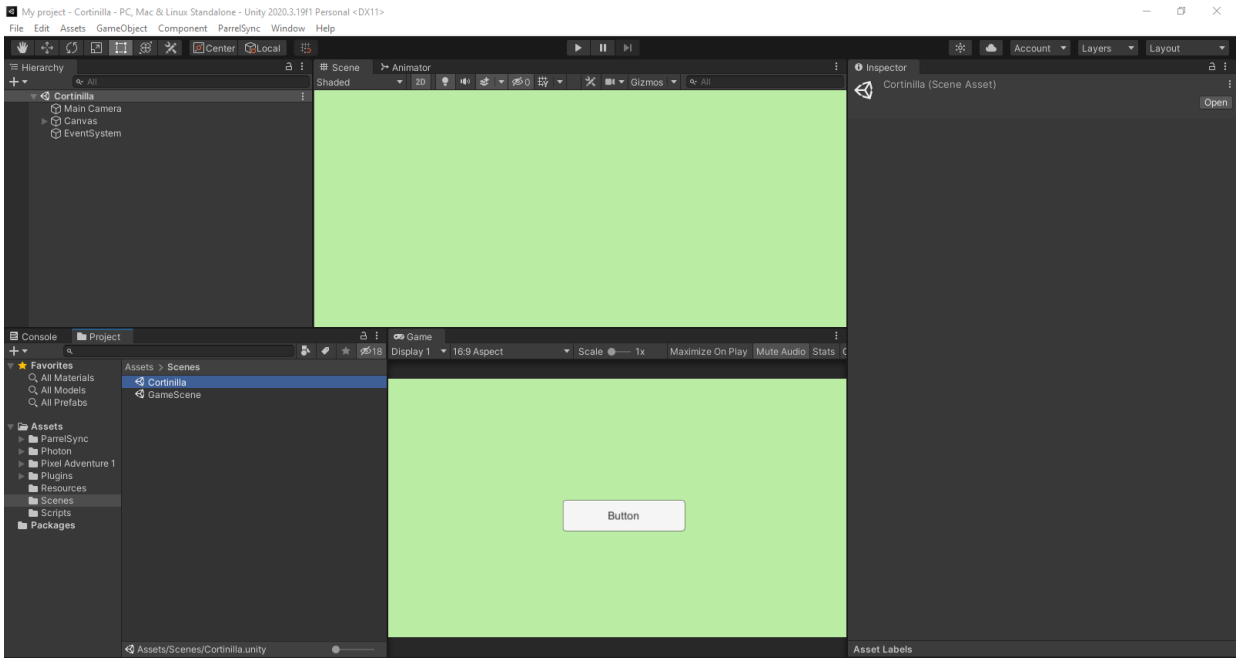
Una imagen final del juego sería:



2. Creación de escenarios

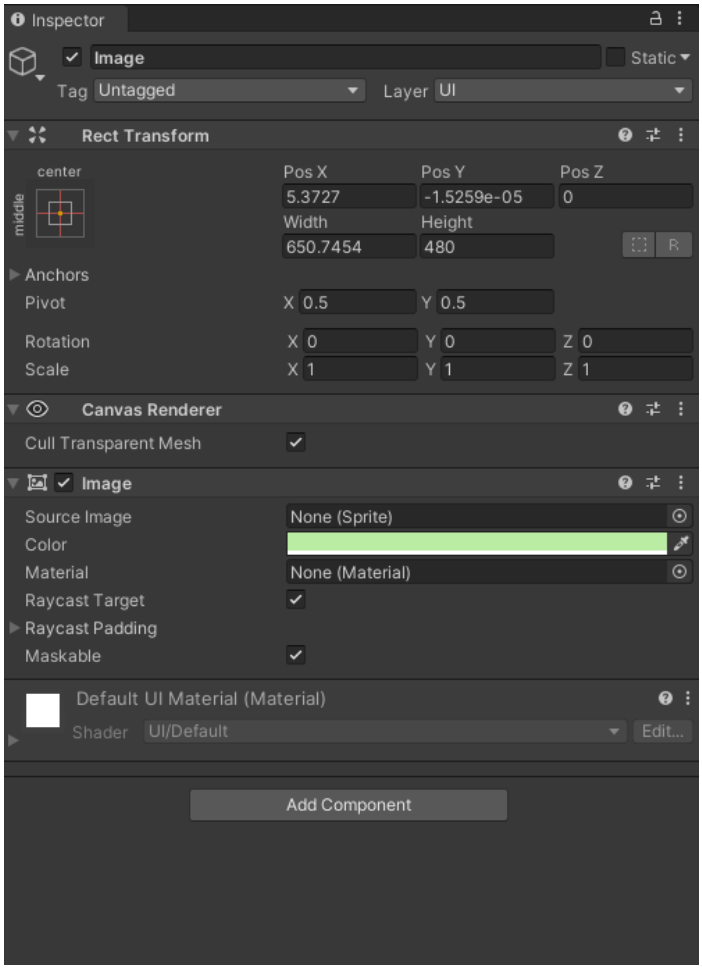
Primero creamos un juego 2D vacío. En este primer paso debemos crear el proyecto con dos escenas. Una primera escena para realizar la conexión al servidor y esperar a que el compañero jugador se conecte. La segunda escena es la que desarrolla el juego en sí.

Crearemos dos escenas, una primera que llamaremos Cortinilla y otra escena que llamaremos GameScene:

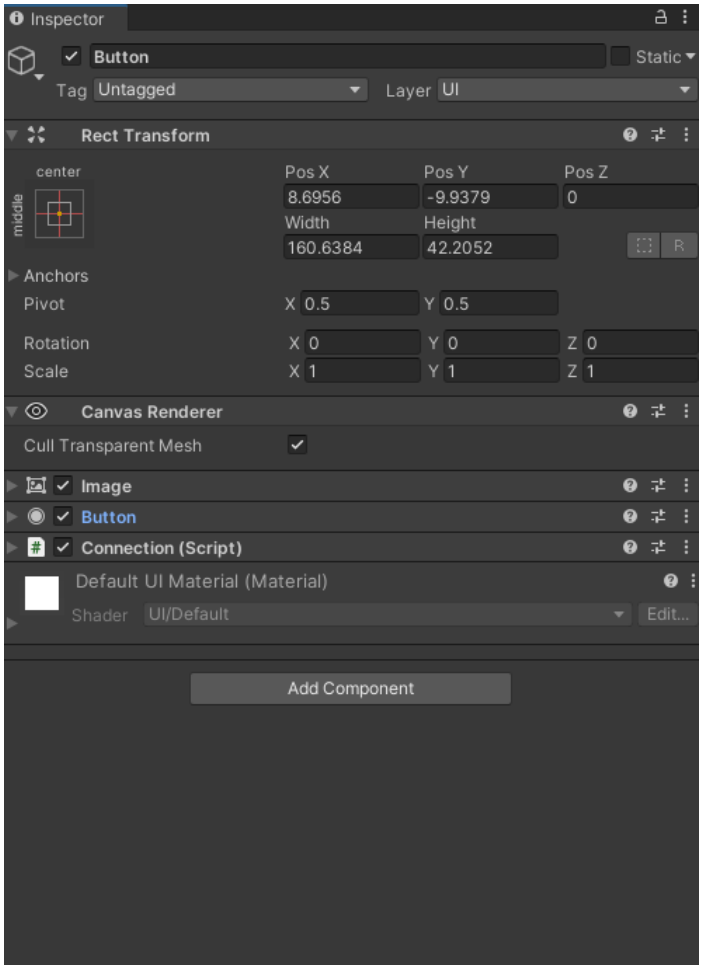


a. Escena Cortinilla

En esta escena vamos a crear un botón dentro de un canvas. Primeramente vamos a poner una imagen de fondo:



También le vamos a añadir un botón al canvas, con el texto Conexión.

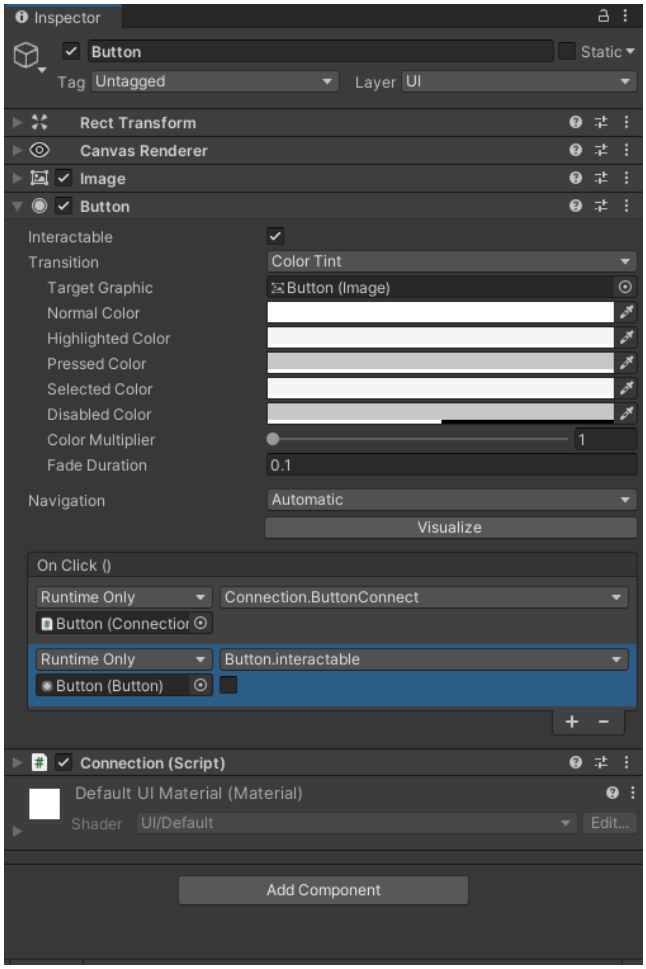


Tendremos que programar el funcionamiento del botón correspondiente, para creamos un directorio que se llame Scripts y creamos un archivo c# con el siguiente código:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

//Librerías de Photon
using Photon.Pun;
using Photon.Realtime;

Script de Unity (1 referencia de recurso) | 0 referencias
public class Connection : MonoBehaviourPunCallbacks
{
    Mensaje de Unity | 0 referencias
    private void Start()
    {
        PhotonNetwork.ConnectUsingSettings(); //Nos conectamos con el parámetros definidos.
        PhotonNetwork.AutomaticallySyncScene = true; // Activamos la sincronización de escena.
                                                     // Necesario para el intercambio entre escenas.
    }


    // Método para conectarse al máster
    override
    3 referencias
    public void OnConnectedToMaster()
    {
        print("Conectado al master !!!!");
    }


    //Método de conexión con el máster con el botón
    0 referencias
    public void ButtonConnect()
    {
        RoomOptions options = new RoomOptions() { MaxPlayers = 4};
        PhotonNetwork.JoinOrCreateRoom("room1", options, TypedLobby.Default);


    }
```

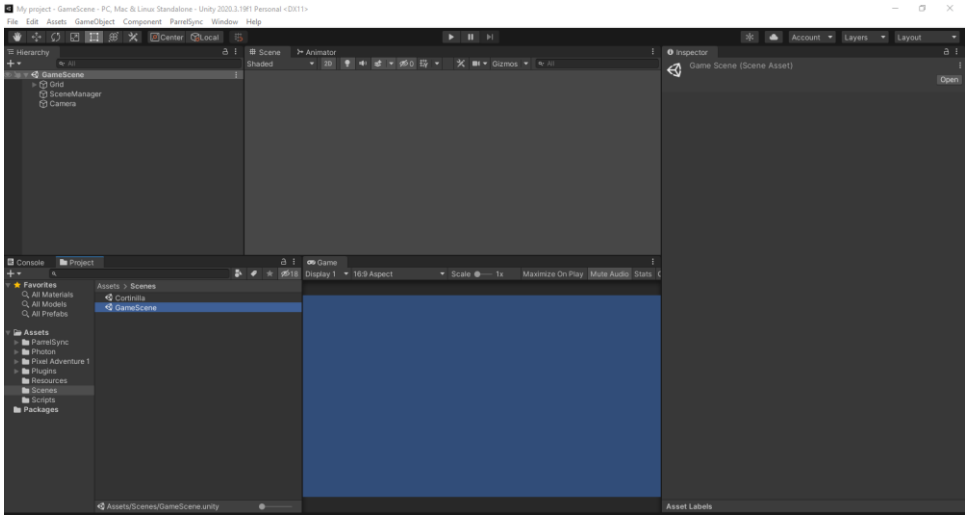
```
    // Método de conexión con una sala
    override
        3 referencias
        public void OnJoinedRoom()
    {
        Debug.Log("Conectada a la sala " + PhotonNetwork.CurrentRoom.Name);
        Debug.Log("Hay..." + PhotonNetwork.CurrentRoom.PlayerCount + " jugadores");

    }
    //Método que controla si pasamos a la siguiente escena si somos más de una persona.
    Mensaje de Unity | 0 referencias
    private void Update()
    {
        if (PhotonNetwork.IsMasterClient && PhotonNetwork.CurrentRoom.PlayerCount > 1)
        {
            //Cargamos siguiente nivel
            PhotonNetwork.LoadLevel(1);
            Destroy(this);
        }
    }
}////////
```

Después de haber creado este script, tenemos que asignárselo al botón que hemos creado.



b. Escena GameScene

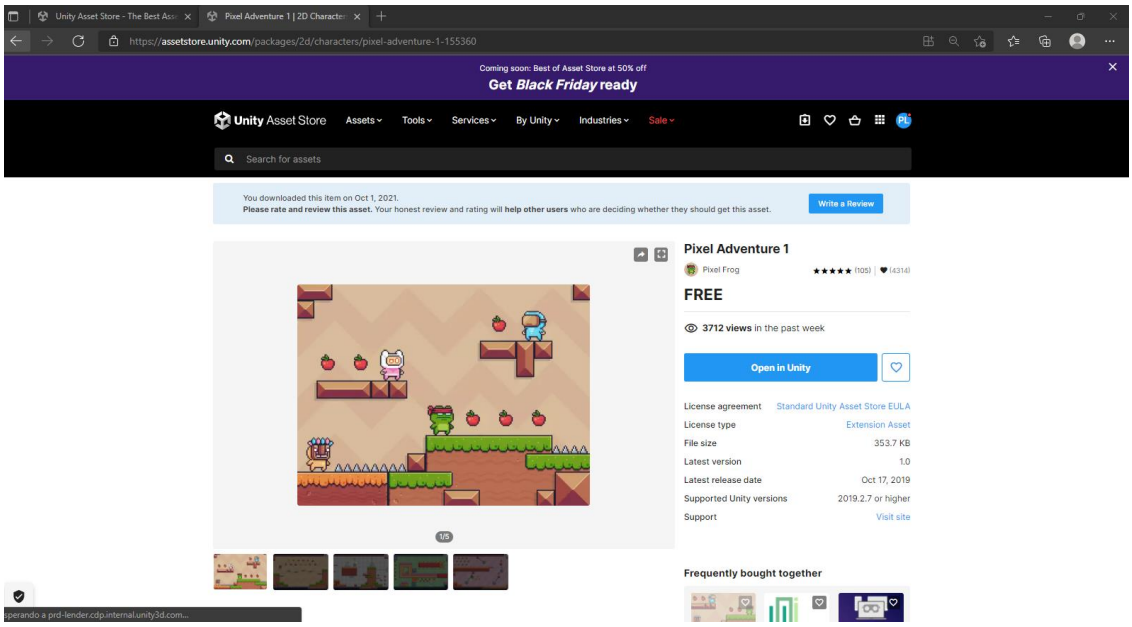
Crearemos otra escena que se llame GameScene.

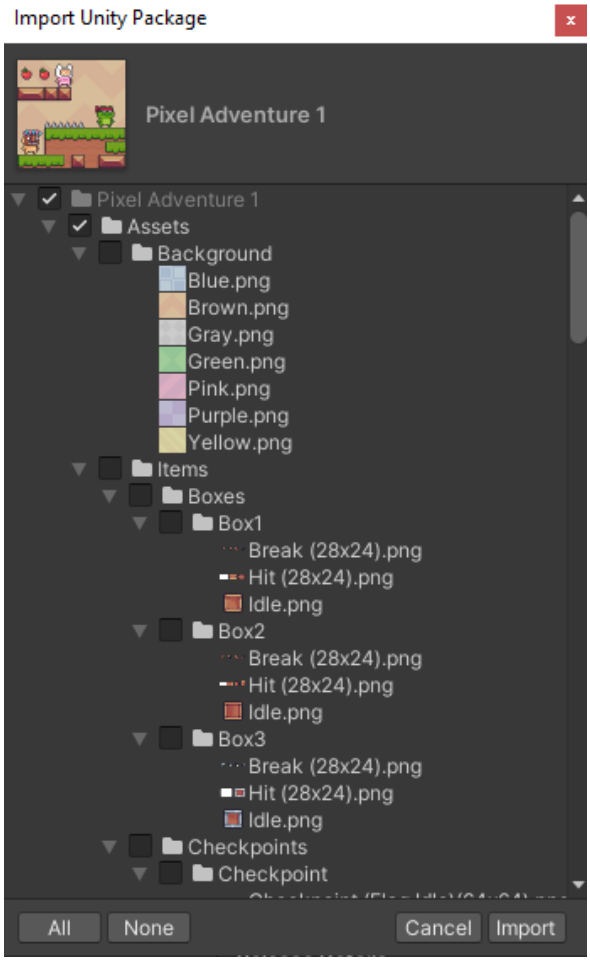


Antes de continuar, debemos pensar qué elementos son susceptibles de ser sincronizados, es decir, si el fondo es estático ¿para qué lo vamos a sincronizar? ¿el suelo? Inclusive una plataforma que se mueve siempre desde un mismo origen a un destino determinado desde el primer momento, no deberemos incluirlo en la sincronización.

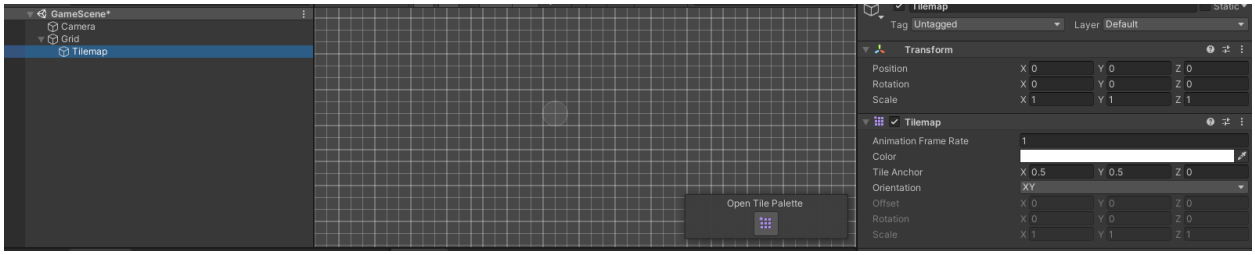
Vamos a crear en la escena el fondo y alguna plataforma que no se sincronicen:

Primeramente, nos vamos a ir a la tienda de Unity para utilizar diversos elementos, importaremos el siguiente:

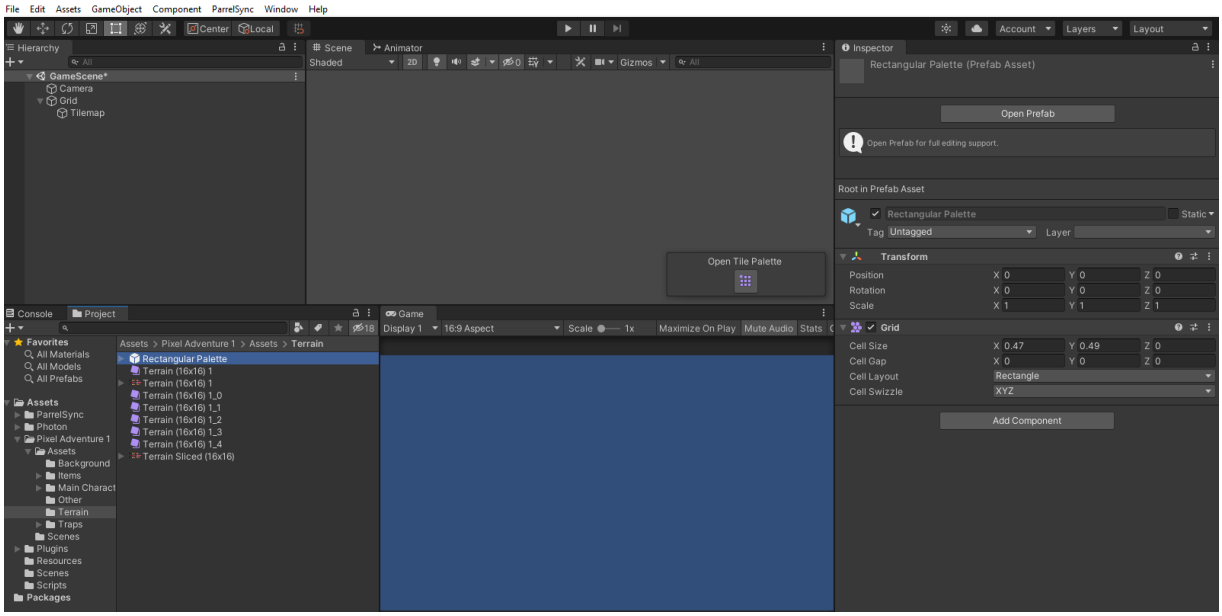




Ahora creamos un Tilemap 2D rectangular.

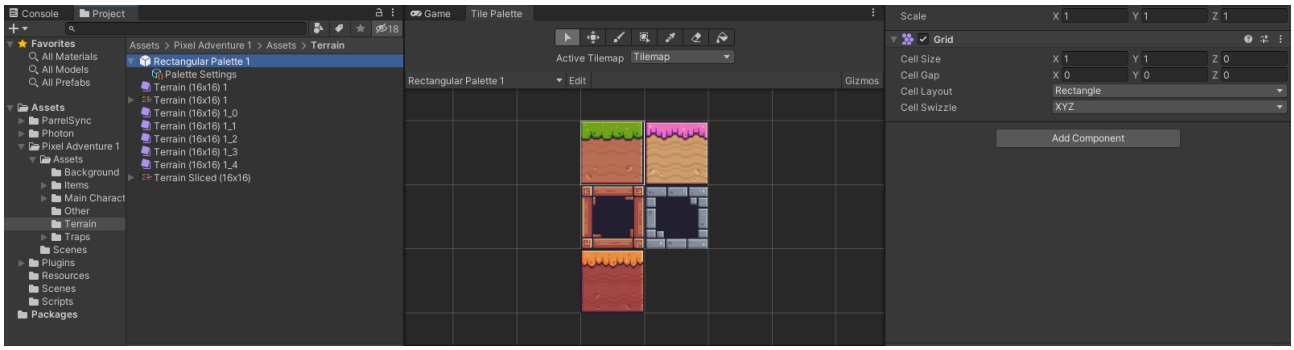


Ahora cremos un un 2D-> tile palette -> Rectangular dentro de la carpeta Pixel Adventure 1 -> Assets -> Terrain

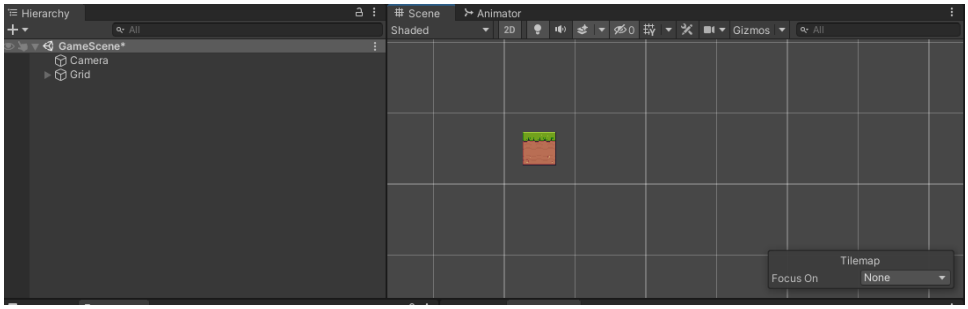


En la misma carpeta tenemos el conjunto que se llama Terrain (16x16) 1 lo vamos a editar con Sprite Editor, pero antes Ponemos el Sprite Mode => Multiple

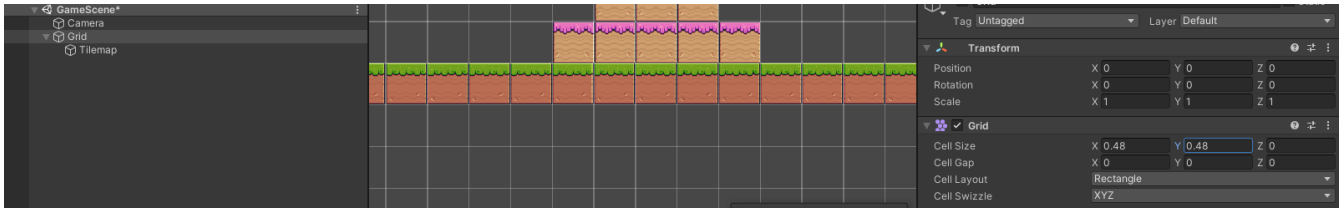
Vamos a ir seleccionando cuadrados de lo que nos interese. Vamos a coger unos 5, una vez que los tengamos recortados los arrastramos a nuestro tile palette. Y quedaría de la siguiente forma:



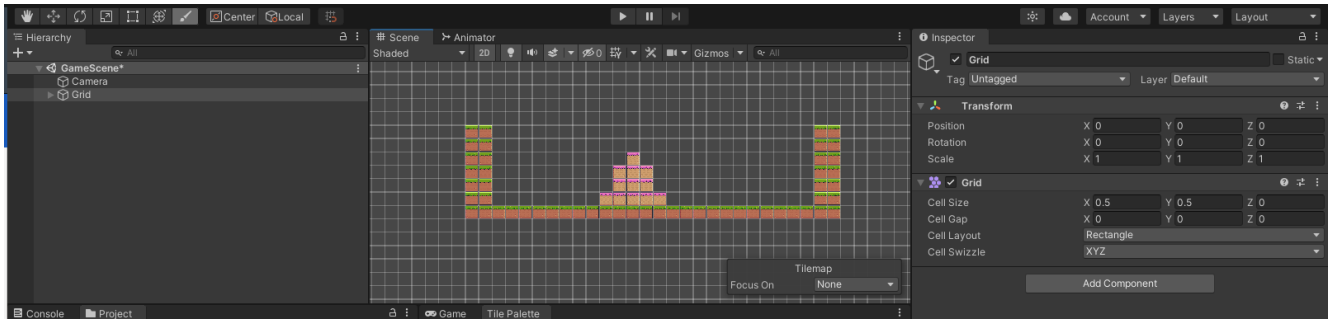
Ahora podemos ir arrastrando a nuestra escena, pero nos vamos a dar cuenta de que es demasiado pequeño cada una de los recuadros que hemos hecho:



Tendremos que ajustar el grid para que funcione correctamente:



Y ahora sí que diseñamos:

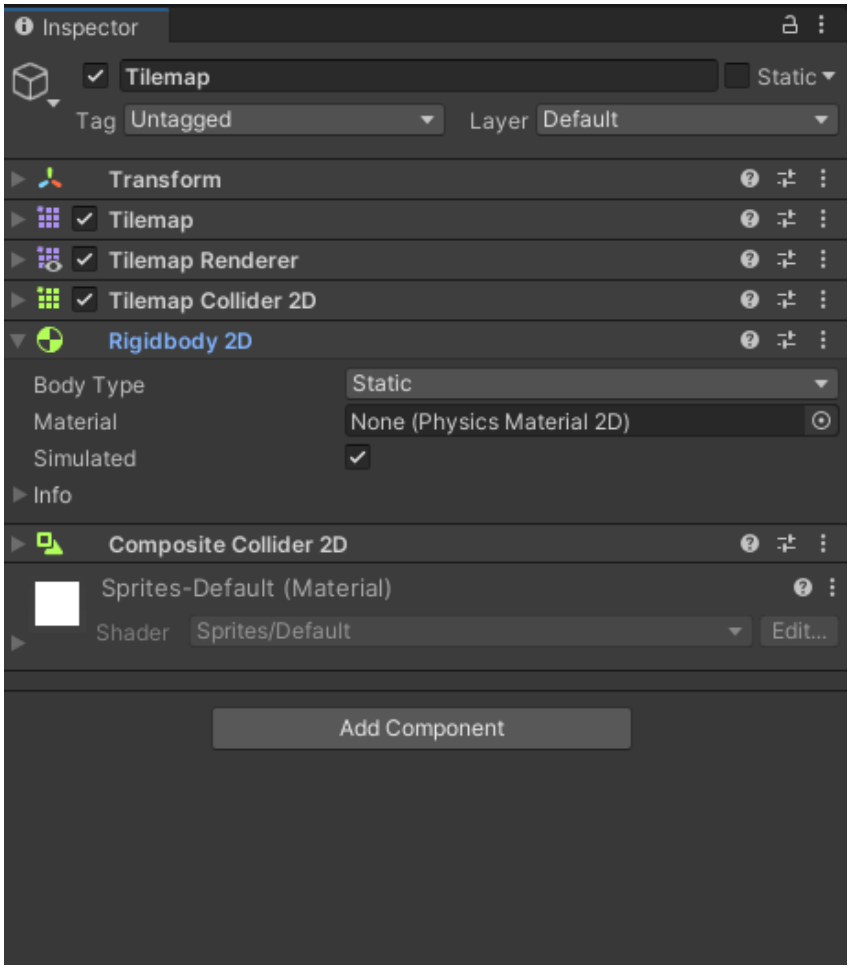


Añadimos varios componentes:

Tilemap render

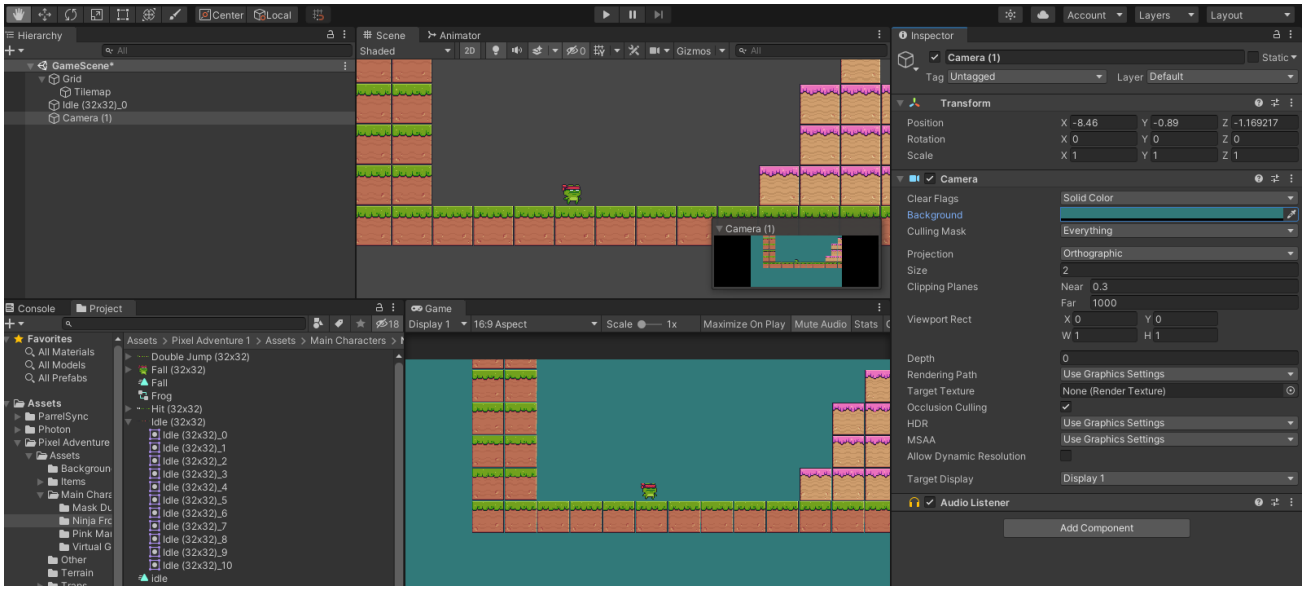
Tilemap Collider 2D

Composite Collider 2D (para unir diversos collider) y tipo de Body static



4. Personaje Ninja Frog y sus animaciones

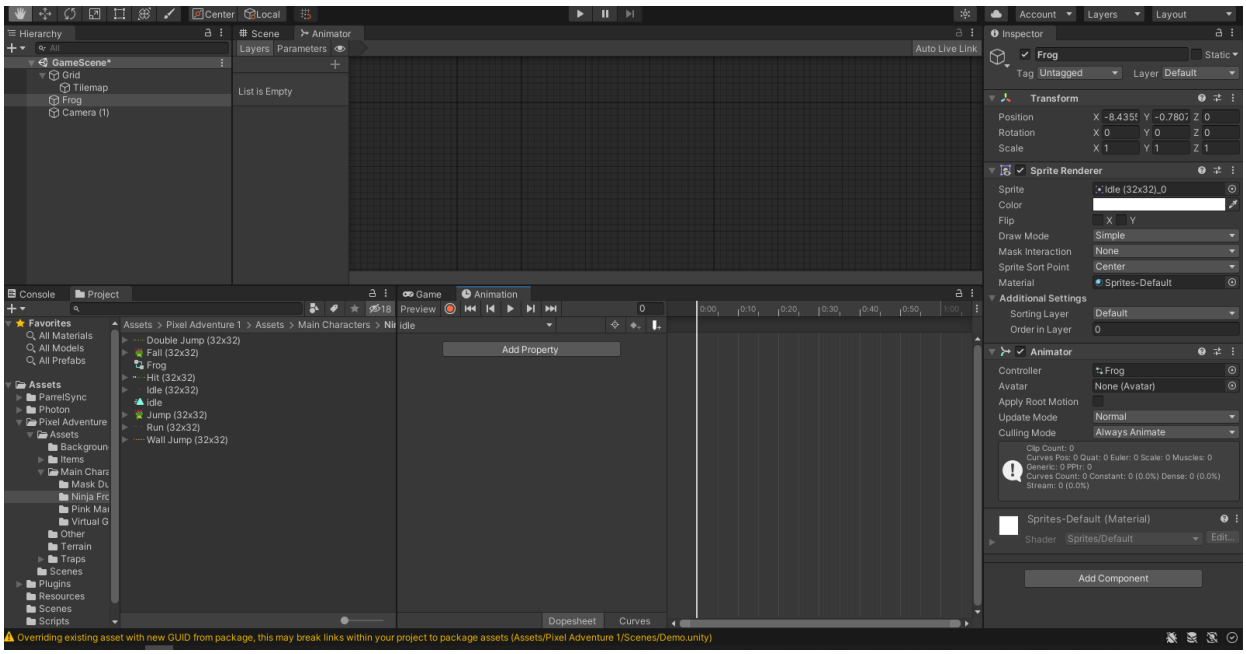
Vamos a empezar con el personaje Ninja Frog. Vamos a añadir el personaje ninja frog idle. También vamos a añadir una cámara y lo vamos a configurar de la siguiente manera: Ortogonal, fondo (que no sea sky), y con tamaño 2.



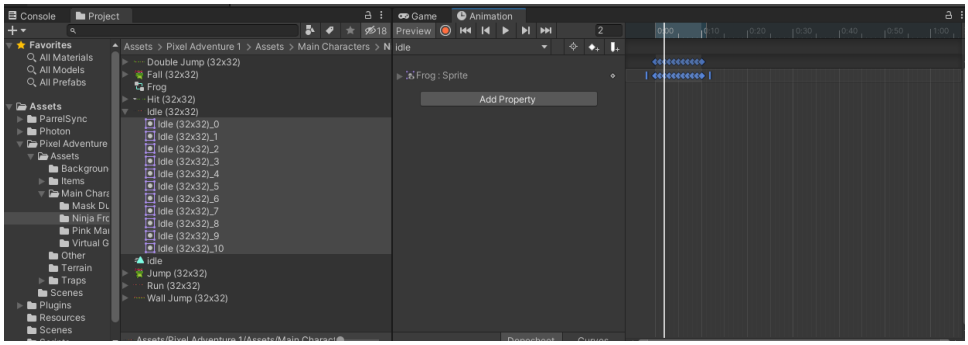
Y ahora vamos a configurar el personaje, tenemos que crearle sus animaciones y su script de movimiento.

Abrimos la ventana “Animator” y “animation”.

Cuando creamos una animación, nos crea directamente un animation Controller. Vamos a empezar con el idle (y así lo vamos a llamar):

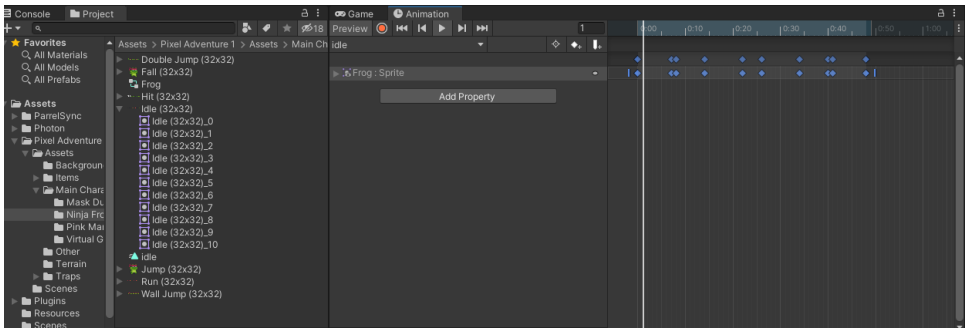


Arrastramos desde el proyecto al Animation el idle:

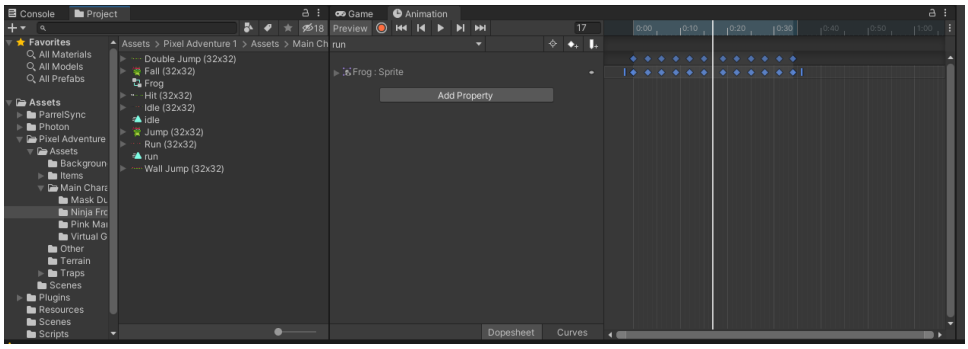


Si le damos al play, podemos ver que va demasiado rápido.

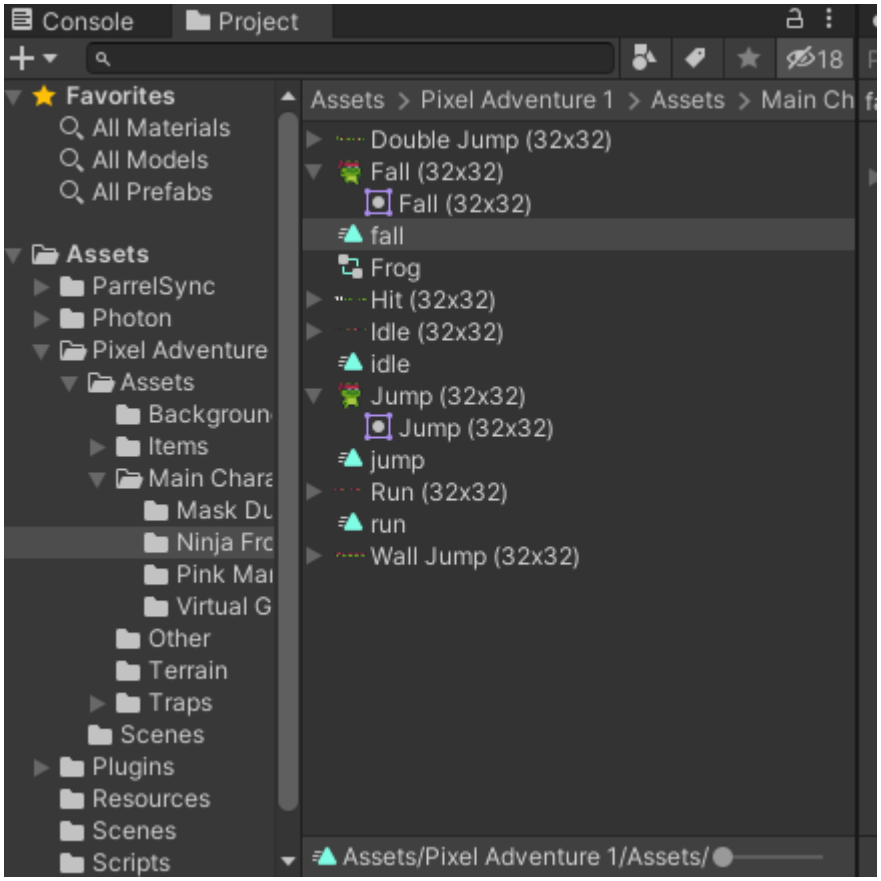
Podemos hacer que la animación dure más en el tiempo:



Creamos otra animación para correr run. Create new clip.



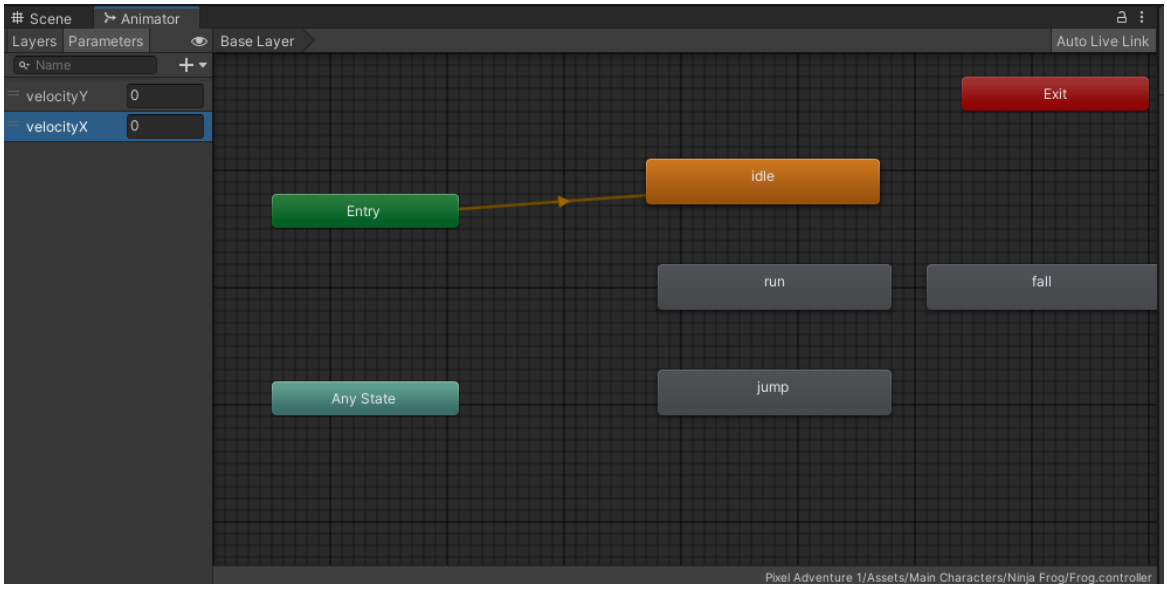
Y hacemos otra de salter jump y su fall.



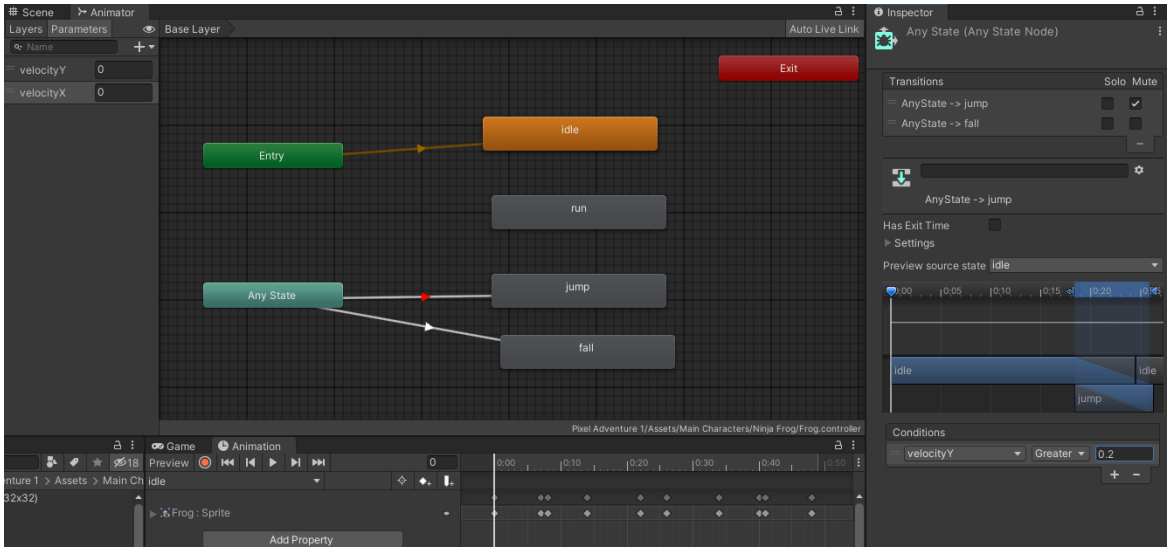
Ahora tenemos que configurar el Animator para que todo funcione correctamente.

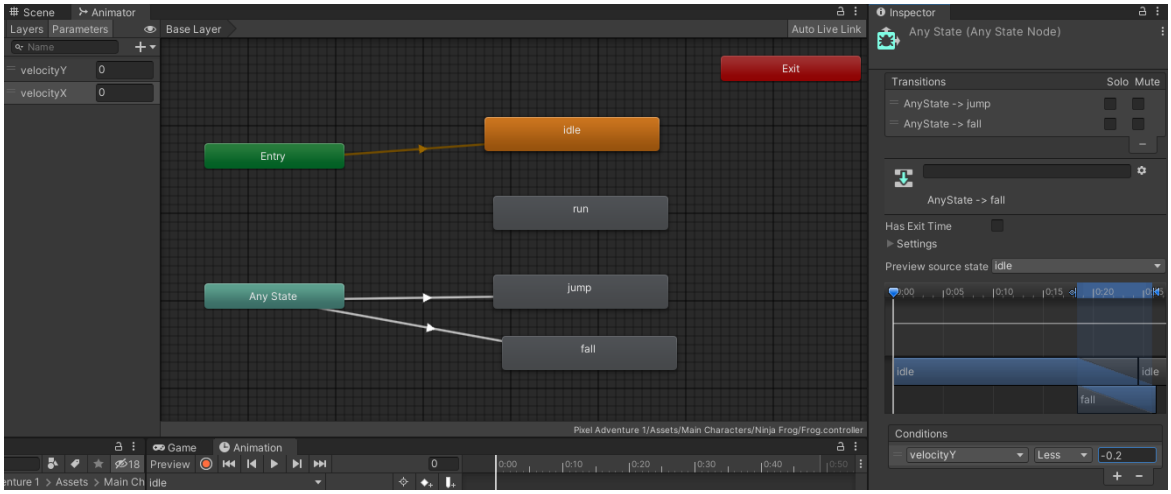
Primero, configurar los parámetros para hacer las transiciones entre uno y otros:

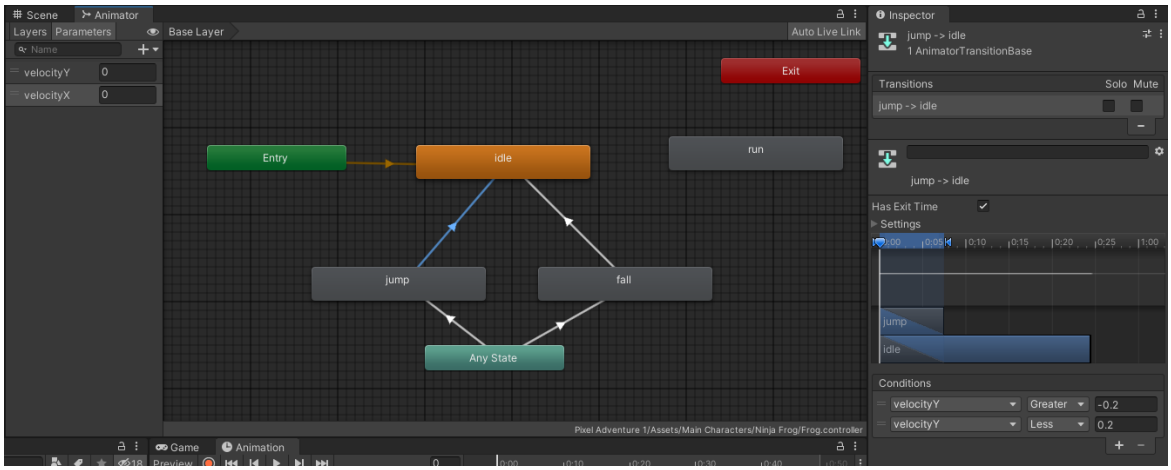
Creamos dos parámetros de tipo float y también con la siguiente distribución:

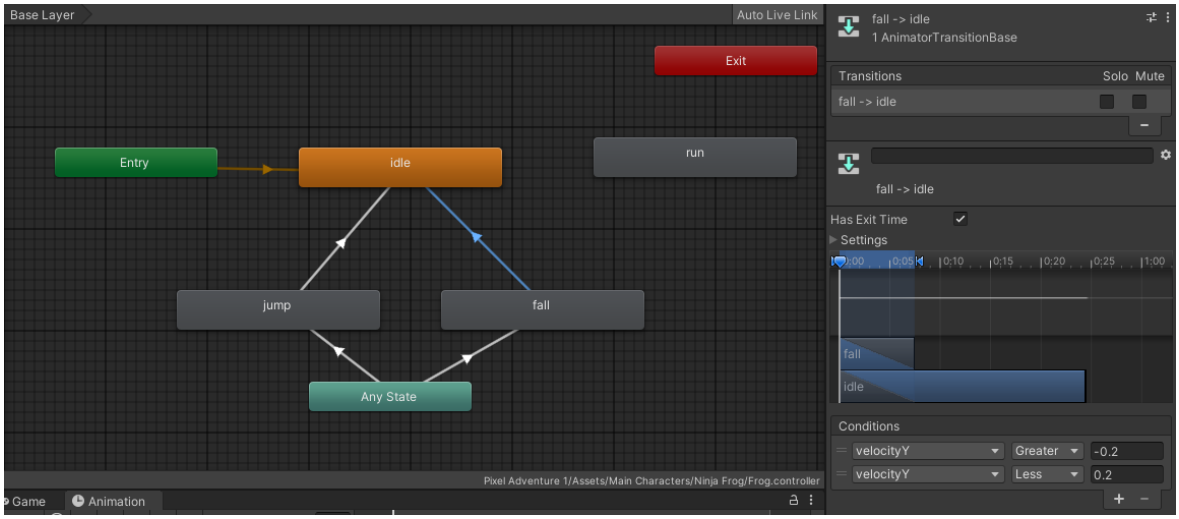


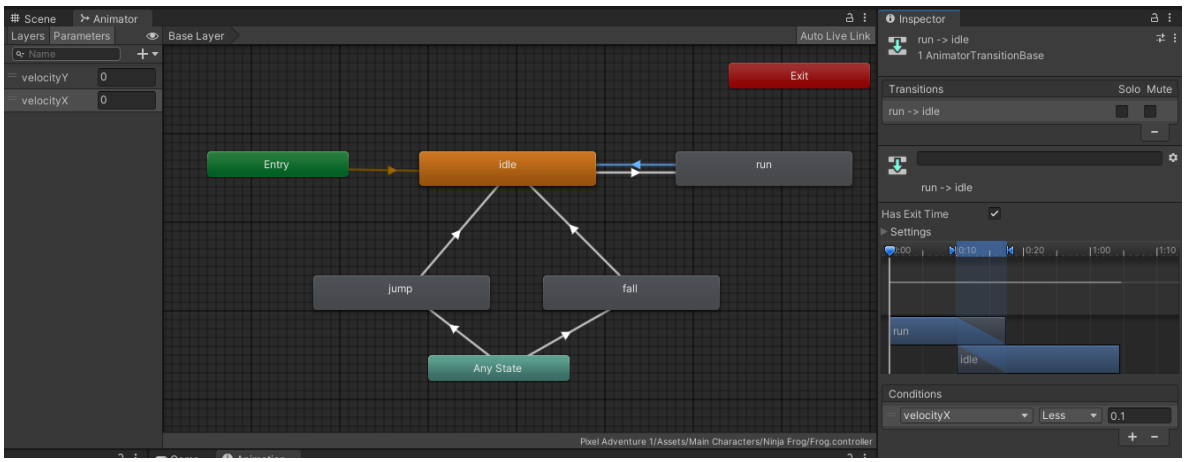
Creamos las transiciones con los parámetros:


Scene
Animator
Layers
Parameters
Base Layer
Auto Live Link
velocityY 0
velocityX 0
Exit
Entry
idle
run
Any State
jump
fall
Pixel Adventure 1/Assets/Main Characters/Ninja Frog/Frog.controller
Game
Animation
Preview
Frog : Sprite
Add Property
Inspector
Any State (Any State Node)
Transitions
Solo
Mute
AnyState -> jump
AnyState -> fall
Has Exit Time
Settings
Preview source state idle
Conditions
velocityY Greater 0.2

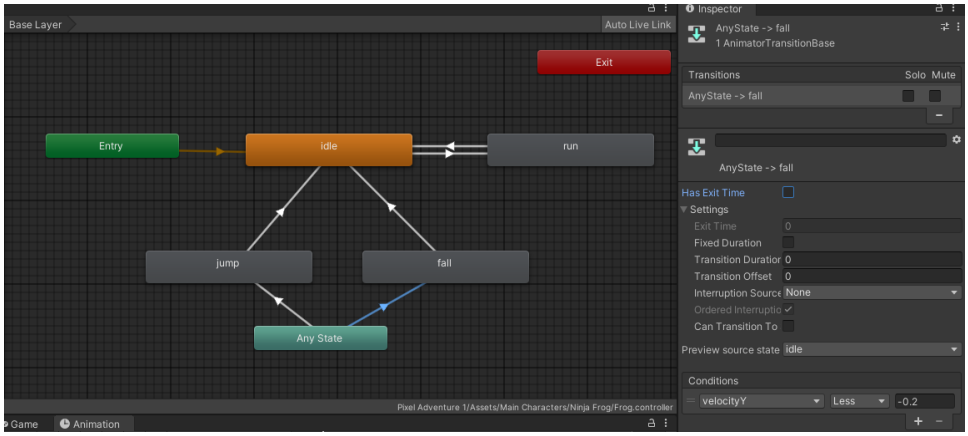

Scene
Animator
Layers
Parameters
Base Layer
Auto Live Link
velocityY 0
velocityX 0
Exit
Entry
idle
run
Any State
jump
fall
Pixel Adventure 1/Assets/Main Characters/Ninja Frog/Frog.controller
Game
Animation
Preview
Inspector
Any State (Any State Node)
Transitions
Solo
Mute
AnyState -> jump
AnyState -> fall
Has Exit Time
Settings
Preview source state idle
Conditions
velocityY Less -0.2

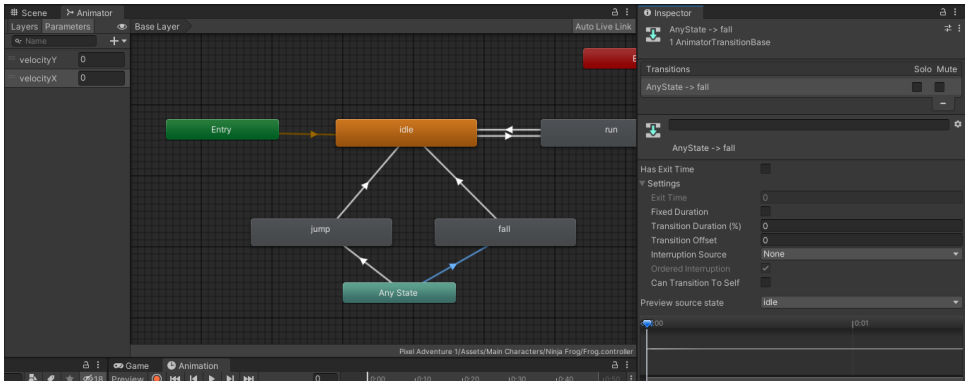

Scene
Animator
Layers
Parameters
Base Layer
Auto Live Link
velocityY 0
velocityX 0
Exit
Entry
idle
run
jump
fall
Any State
Pixel Adventure 1/Assets/Main Characters/Ninja Frog/Frog.controller
Game
Animation
Inspector
jump -> idle
1 AnimatorTransitionBase
Transitions
Solo
Mute
jump -> idle
Has Exit Time
Settings
Conditions
velocityY Greater -0.2
velocityY Less 0.2

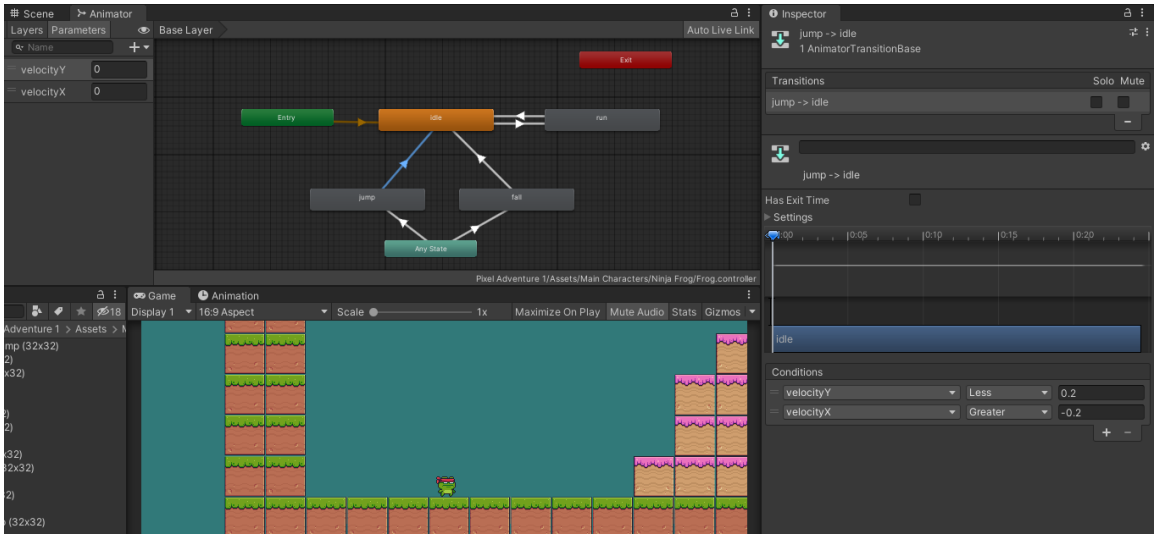


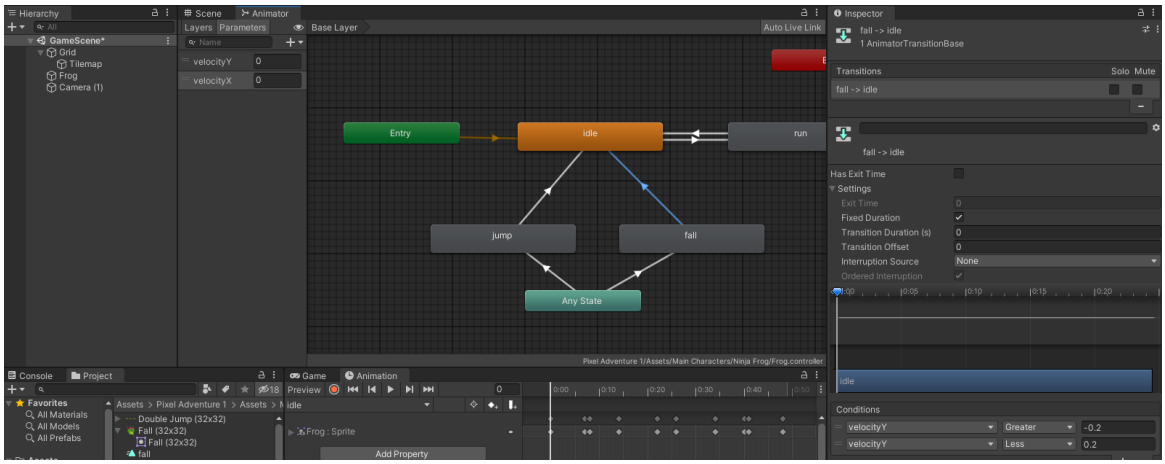


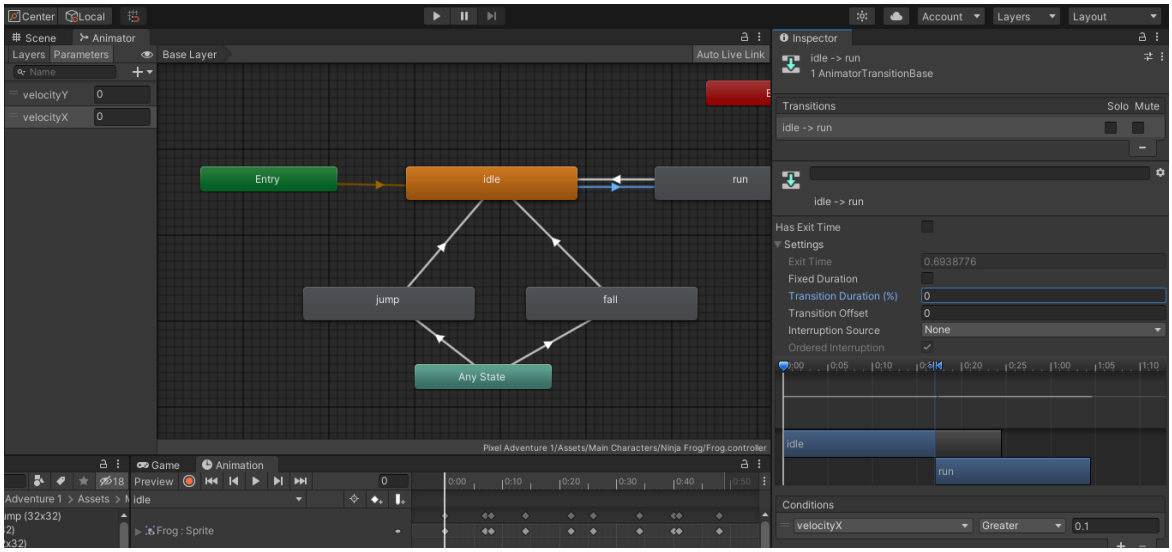
Configuramos las transiciones:

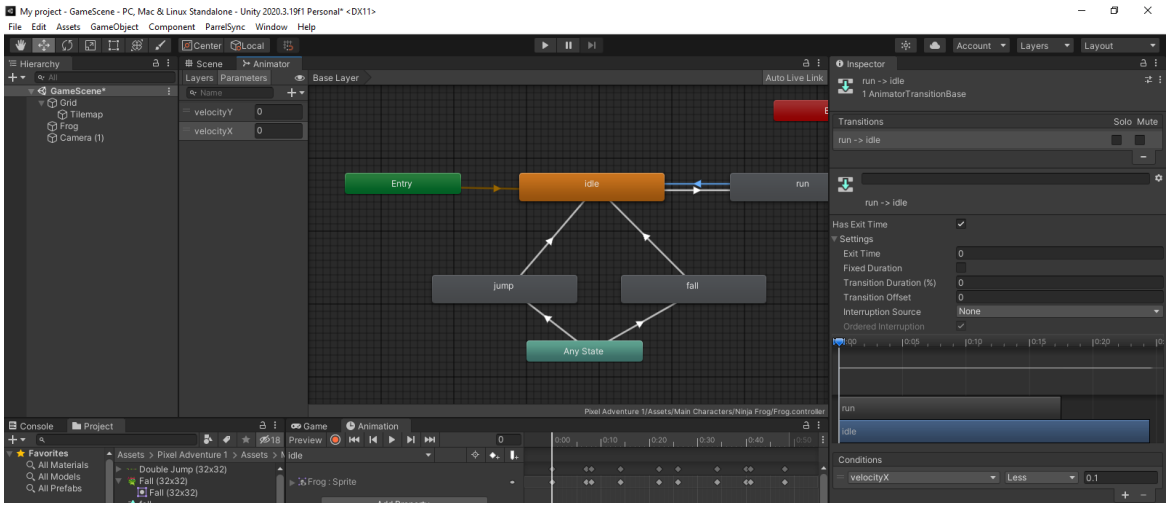



Scene
Animator
Layers
Parameters
Base Layer
Auto Live Link
velocityY
velocityX
Entry
idle
run
jump
fall
Any State
Inspector
AnyState -> fall
1 AnimatorTransitionBase
Transitions
Solo
Mute
Has Exit Time
Settings
Exit Time
Fixed Duration
Transition Duration (%)
Transition Offset
Interruption Source
None
Ordered Interruption
Can Transition To Self
Preview source state
Game
Animation


Scene
Animator
Layers
Parameters
Base Layer
Auto Live Link
velocityY
velocityX
Exit
Entry
idle
run
jump
fall
Any State
Inspector
jump -> idle
1 AnimatorTransitionBase
Transitions
Solo
Mute
Has Exit Time
Settings
Conditions
velocityY
Less
0.2
velocityX
Greater
-0.2
Game
Animation
Display 1
16:9 Aspect
Scale
1x
Maximize On Play
Mute Audio
Stats
Gizmos


Hierarchy
GameScene*
Grid
Tilemap
Frog
Camera (1)
Scene
Animator
Layers
Parameters
Base Layer
Auto Live Link
velocityY
velocityX
Entry
idle
run
jump
fall
Any State
Inspector
fall -> idle
1 AnimatorTransitionBase
Transitions
Solo
Mute
Has Exit Time
Settings
Fixed Duration
Transition Duration (s)
Transition Offset
Interruption Source
None
Ordered Interruption
Conditions
velocityY
Greater
-0.2
velocityY
Less
0.2
Console
Project
Favorites
All Materials
All Models
All Prefabs
Double Jump (32x32)
Fall (32x32)
Frog : Sprite
Add Property
Game
Animation
Preview

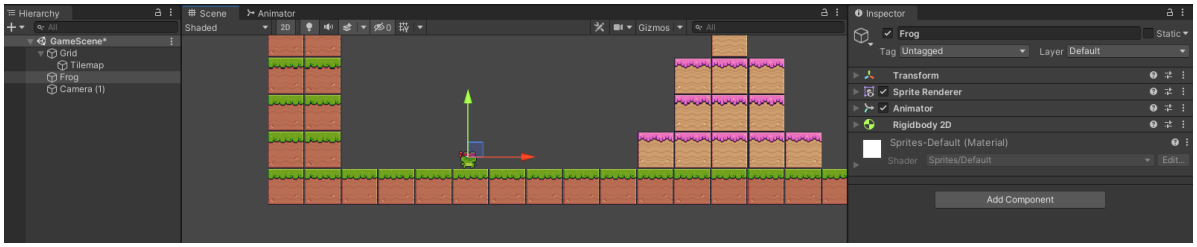




5. Scripts del movimiento del personaje

Una vez que tenemos las animaciones, creamos los scripts que regula el movimiento.

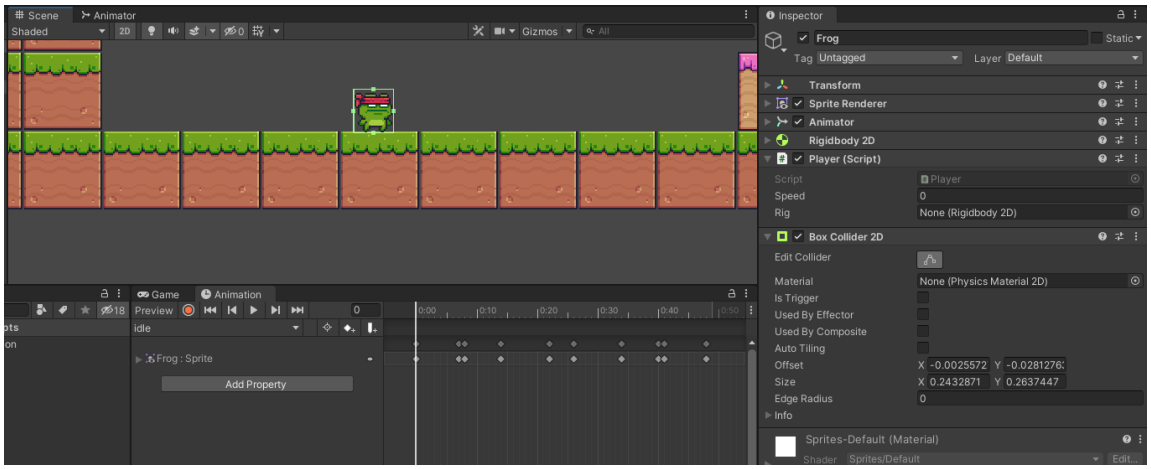
Le añadimos al Frog un Rigidbody 2d para poder moverlo:



Creamos el script, que se va a llamar Player y se lo añadimos a Frog.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

Script de Unity (2 referencias de recurso) | 0 referencias
public class Player : MonoBehaviour
{
    public float speed;
    public Rigidbody2D rig;


    // Start is called before the first frame update
    Mensaje de Unity | 0 referencias
    void Start()
    {
        rig = GetComponent<Rigidbody2D>();
    }

    // Update is called once per frame
    Mensaje de Unity | 0 referencias
    void Update()
    {// Le damos velocidad pero también en el eje y porque sino se quedaría parado.
        rig.velocity = (transform.right * speed * Input.GetAxis("Horizontal")) + (transform.up * rig.velocity.y);
    }
}
```

Ya podemos probarlo pero debemos ponerle un collider ajustado para que no se caiga 😉



Y ahora sí que lo podemos probar, dándole un valor a speed, por ejemplo, un valor de 1

Se puede mover con las flechas izquierda y derecha o las teclas a y d.

Añadimos ahora a nuestro script para que gire (flip) el personaje:

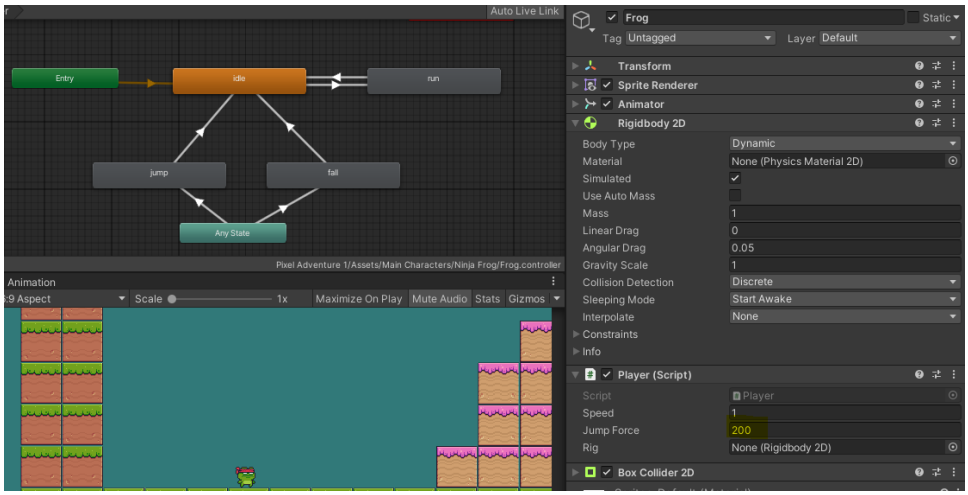
```
// Update is called once per frame
Mensaje de Unity | 0 referencias
void Update()
{// Le damos velocidad pero también en el eje y porque sino se quedaría parado.
    rig.velocity = (transform.right * speed * Input.GetAxis("Horizontal")) + (transform.up * rig.velocity.y);

    if (rig.velocity.x > 0.1f) //Cambiamos la imagen de movimiento
        GetComponent<SpriteRenderer>().flipX = false;
    else if(rig.velocity.x<0.1f)
        GetComponent<SpriteRenderer>().flipX = false;

}
```

De la mecánica del personaje nos queda el salto.

Nos vamos al script de Player y añadimos:

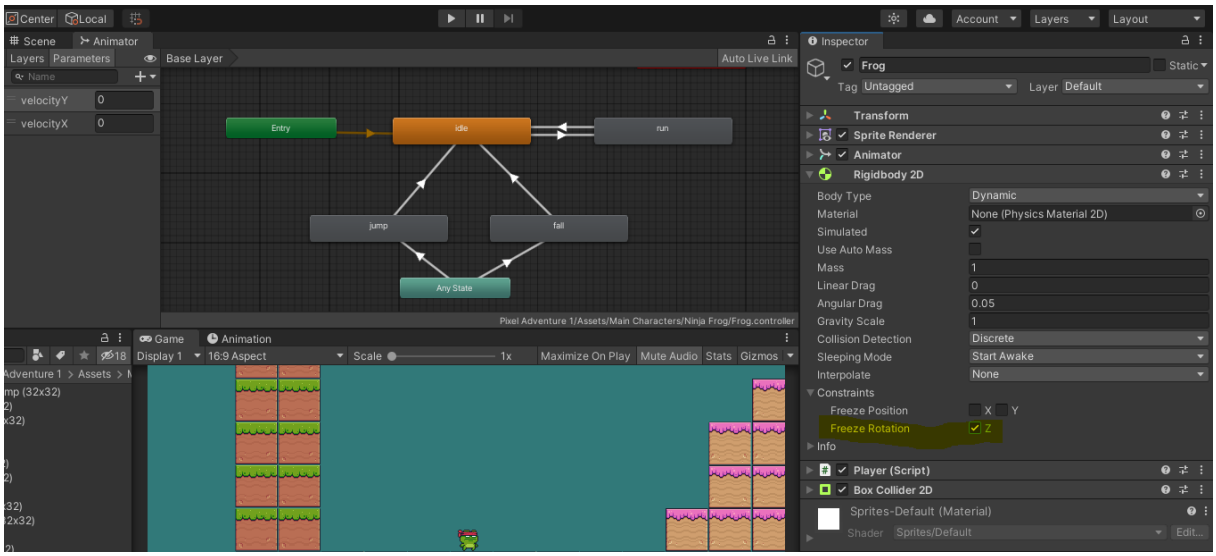
```
//añadimos el salto
if (Input.GetButtonDown("Jump"))
{
    rig.AddForce(transform.up * jumpForce);
}
```

Junto con la variable speed, creamos otra variable llamada jumpForce y le damos un valor en Unity (puesto que es una variable pública). Le damos un valor de 200 y probamos que funcione.



Vamos a poner también la cámara dentro de Frog para que le siga.

Como habrás podido comprobar, el Rigidbody hace que gire el personaje. Lo bloqueamos para que no se gire.



Nos queda ahora sincronizar las animaciones. Para esto vamos a ir a nuestro script.

Primeramente, creamos una variable privada para contener al Animator.

```
private Rigidbody2D rig;
private Animator anim;

0 referencias
private void Start()
{
    rig = GetComponent<Rigidbody2D>();
    anim = GetComponent<Animator>();
}
0 referencias
```

También añadimos al método update:

```
    //añadimos el salto
    if (Input.GetButtonDown("Jump"))
    {
        rig.AddForce(transform.up * jumpForce);
    }
    //Añadimos la animación.
    anim.SetFloat("velocityX", rig.velocity.x);
    anim.SetFloat("velocityY", rig.velocity.y);
}
```

Como puedes ver, hacia la izquierda no quiere andar y eso es porque no contemplamos números negativos en el Animator.

Esto lo podemos solucionar con una función matemática para coger el valor absoluto:

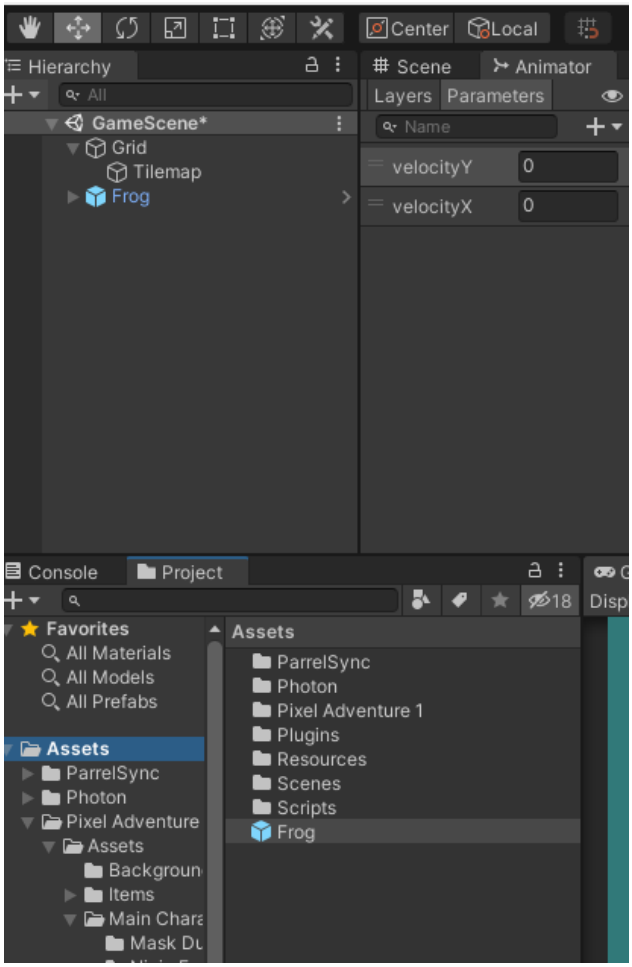
anim.SetFloat("velocityX", Mathf.Abs(rig.velocity.x));

¡Enhorabuena si has llegado hasta aquí! Ahora falta configurar el otro adversario/jugador y dotarle de sincronismos entre ambos.

6. Personaje pequeño Buzo y sus animaciones

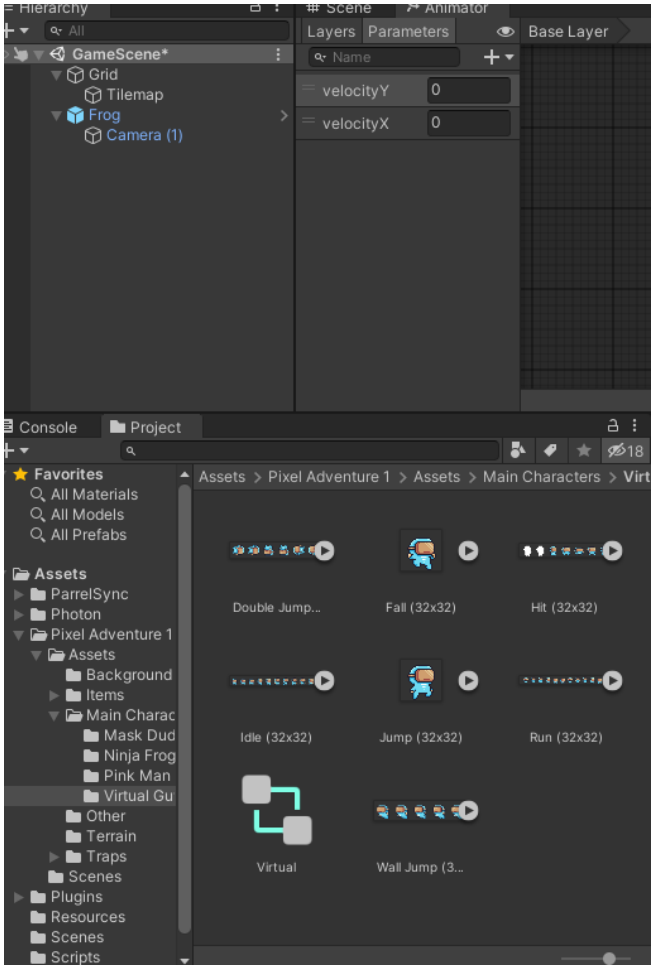
La configuración del segundo personaje va a ser muy sencillo, solamente va a cambiar el Animator.

1º vamos a hacer un prefab. Arrastrando y soltando:

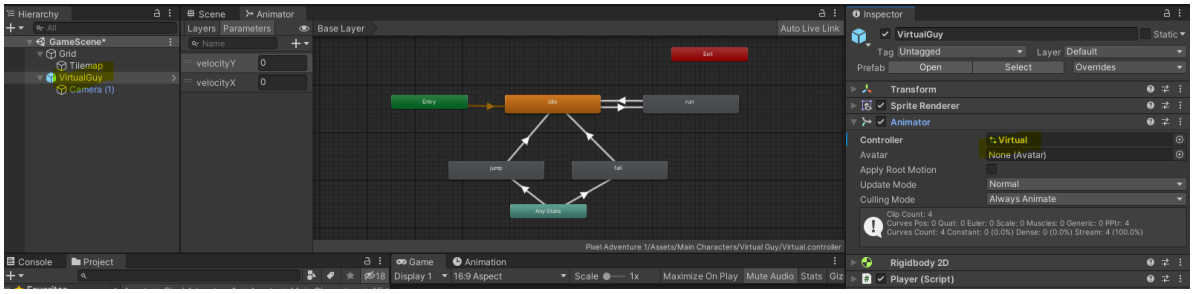


El usuario que vamos a coger es el Virtual Guy. Que es un buzo.

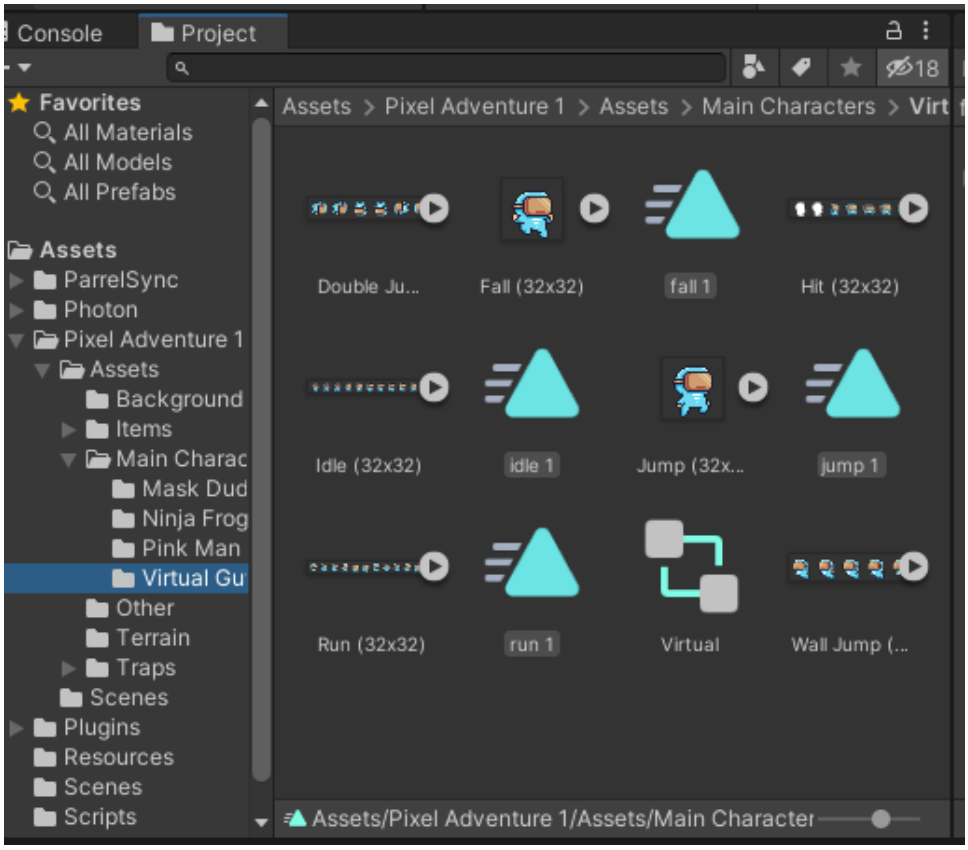
Nos vamos a su carpeta y vamos a crear otro Animator Controller. Vamos a duplicar el que tenemos de la rana y lo copiamos en nuestra carpeta de Virtual Guy. Llamando al Animator: Virtual



También vamos a cambiar el nombre del objeto llamándolo VirtualGuy y le vamos a poner su Animator.

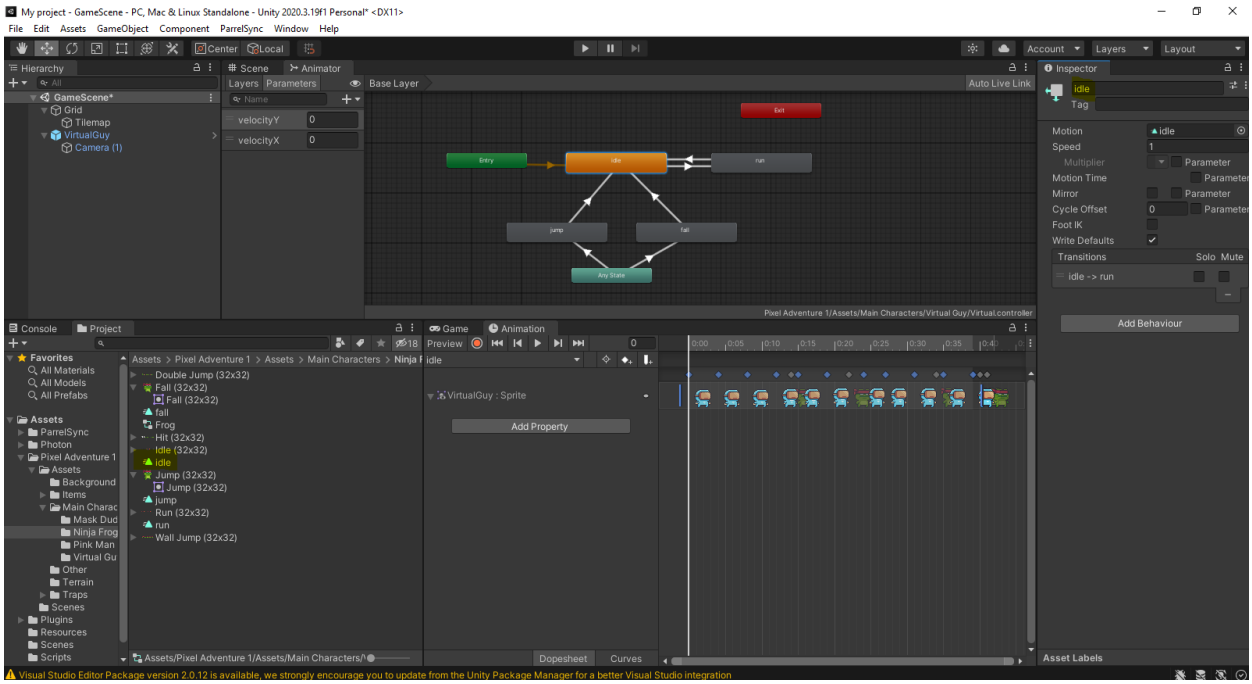


Y es a este Animator al que tengo que cambiar las animaciones. Lo que vamos a hacer es duplicarlas en la rana y pasarlas al buzo.

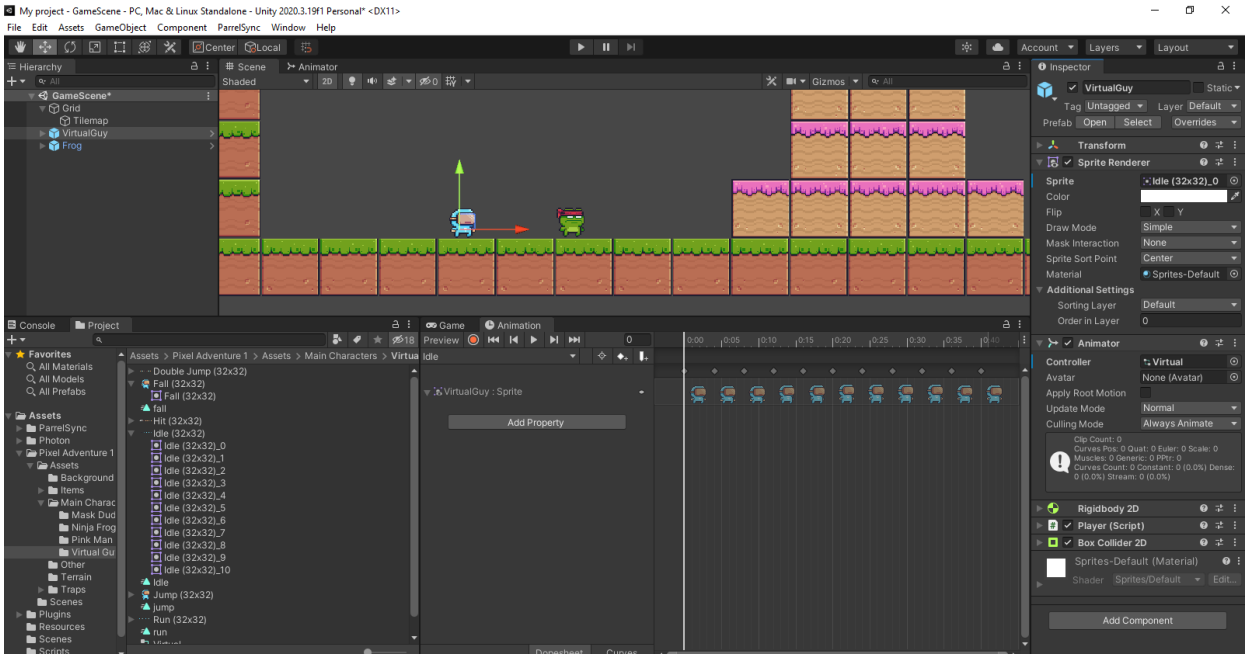


Les quitamos el 1 que sobra en su nomenclatura.

Luego cambiamos la referencia en el animator, para que modifiquemos el de la rana. Tenemos que ir a cada estado, coger y soltar:



Ahora podemos modificar las animaciones tranquilamente. También le tenemos que cambiar la imagen. Puede que sea interesante poner el buzo para seleccionarlo y no tocar la rana.



Una vez probadas las animaciones ya podemos guardar este personaje como un prefab, un prefab original.

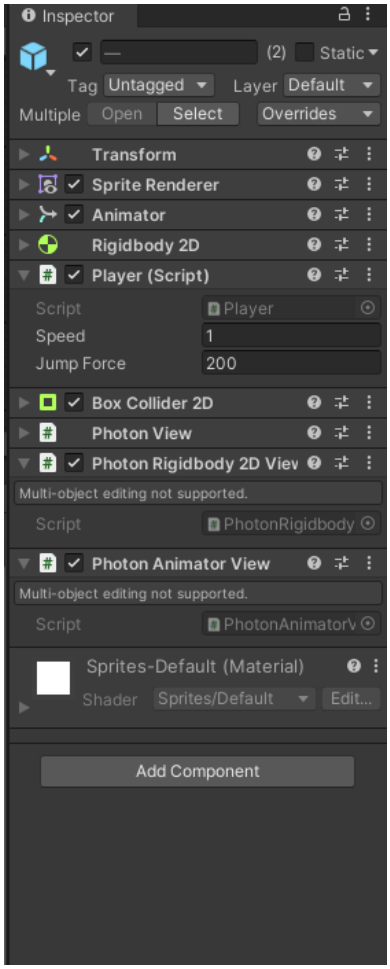
Si y ejecuto vamos a mover a los dos a la vez.

7. Configuración Escena Multijugador

Solamente vamos a dejar en la escena lo que no vamos a sincronizar. Por ejemplo, el escenario es estático y podríamos ponerlo en estático.

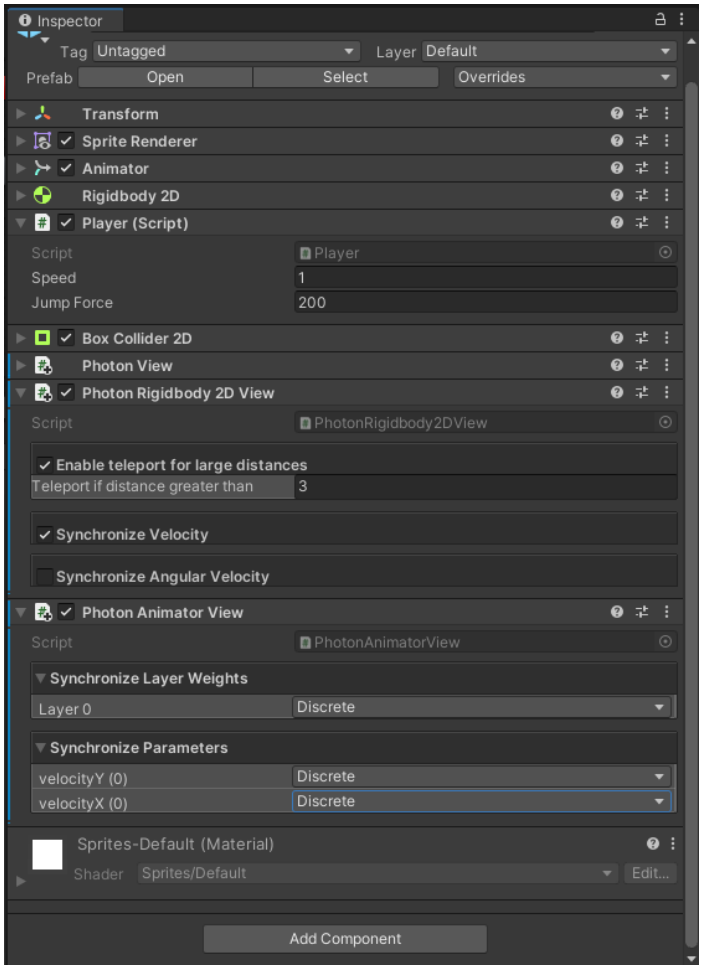
Para sincronizar la rana y el buzo, tenemos que ponerle los componentes de Photon:

- Photon View
- Photon Rigidbody 2D View
- Photon Animator View



Modificamos los parámetros de Photon Animator View. Si es discreta se va a sincronizar a en tiempo discreto. La sincronización discreta viene a ser menos difícil para el rendimiento. También vamos habilitamos los valores de larga distancia.

Ponemos los siguientes valores:



Para el buzo también ponemos los mismos parámetros.

Aplicamos los cambios en su prefab. En el Inspector, en override. Una vez salvados, los eliminamos para guardar la escena.

Quien sincroniza es el propietario del objeto. Si soy el jugador 1 que ser propietario, es decir, cada jugador debe instanciar. Pero instanciándolo con Photon, para que el máster lo sepa en el servidor.

Creamos un GameObject vacío al que le vamos a asignar un script. Este GameObject lo vamos a llamar SceneManager y al script los vamos a llamar GameManager que es el que va a administrar esta escena.

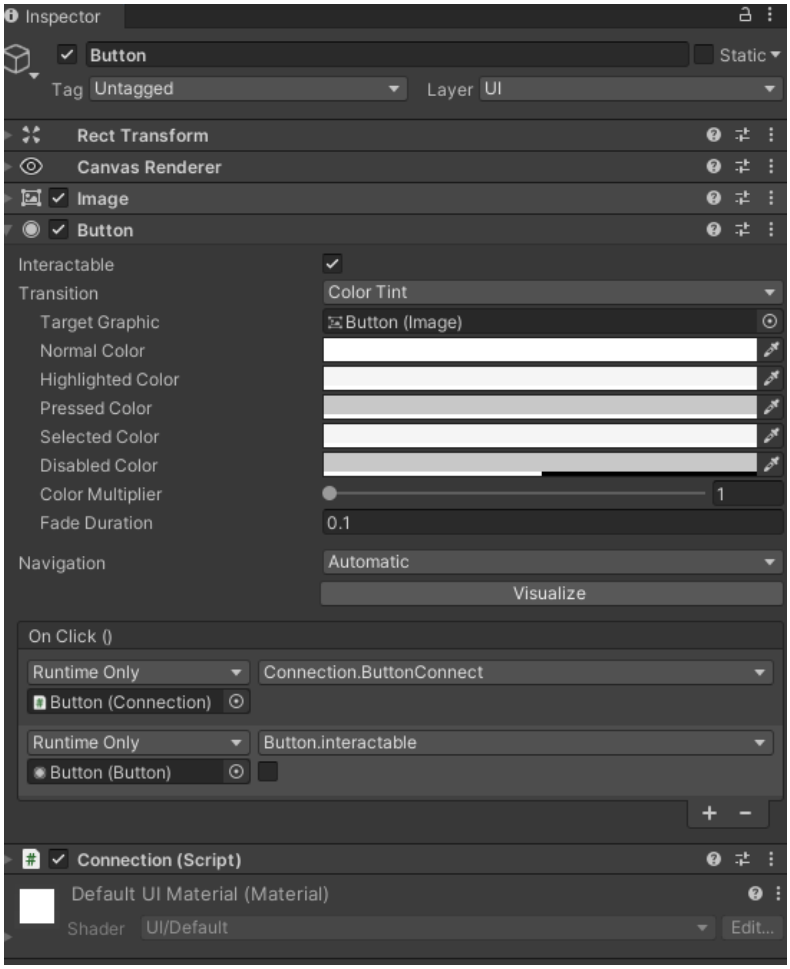
Conseguimos la posición poniendo los jugadores. El script, quedaría:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// librerías de Photon
using Photon.Pun;

Script de Unity (1 referencia de recurso) | 0 referencias
public class GameManager : MonoBehaviour
{
    // Start is called before the first frame update
    Mensaje de Unity | 0 referencias
    void Start()
    {
        // La instanciación la hace Photon.
        // Ojo el nombre lo busca en una carpeta que se llama Resources
        // Y es ahí donde tenemos que meter nuestros prefabs
        if (PhotonNetwork.IsMasterClient)//Si soy el máster, soy el jugador 1.
                PhotonNetwork.Instantiate("Frog", new Vector3(-9,-8,0) , Quaternion.identity);
        else
            PhotonNetwork.Instantiate("VirtualGuy", new Vector3(0, -8, 0), Quaternion.identity);
    }
```

8. Cambio de escena

Tenemos que conectar la escena de conexión y la escena del juego. La escena Cortinilla y GameScene.

Antes de hacer la transición vamos a ponerle al botón de la escena Cortinilla que no sea interactuable cuando se pulsa.

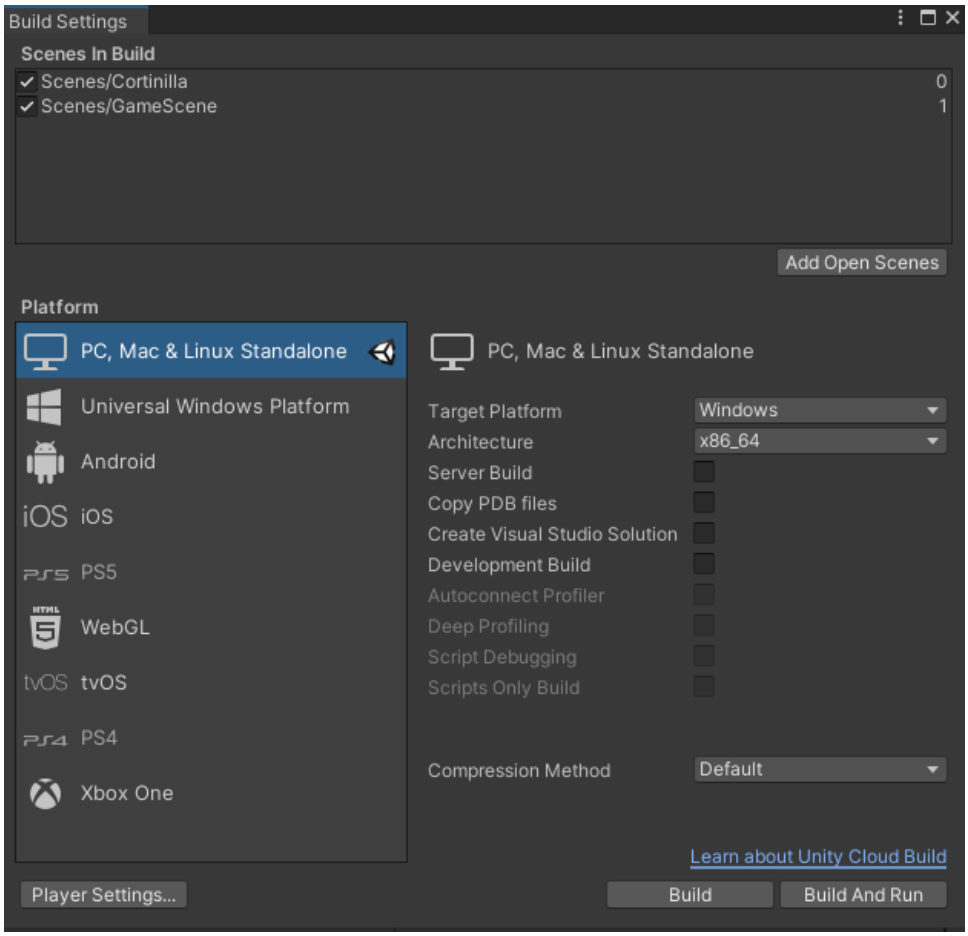


Añadimos al script lo siguiente:

```
Mensaje de Unity | 0 referencias
private void Start()
{
    PhotonNetwork.ConnectUsingSettings(); //Nos conectamos con el parámetros definidos.
    PhotonNetwork.AutomaticallySyncScene = true; // Activamos la sincronización de escena.
                                                 // Necesario para el intercambio entre escenas.
}
```

```
private void Update()
{
    //Si soy el máster y hay más de uno, cargamos la siguiente escena
    if (PhotonNetwork.IsMasterClient && PhotonNetwork.CurrentRoom.PlayerCount > 1)
    {
        //Cargamos siguiente nivel
        PhotonNetwork.LoadLevel(1);
        Destroy(this);
    }
}
```

Debemos incluir la otra escena en el build sttings. Ojo, lo deberémos hacer también en el proyecto clonado.



Si lo ejecutamos los movimientos no se sincronizan correctamente. Esto lo veremos en el siguiente paso.

También nos da un error: “Please ensure there is always exactly one audio listener in the scene.” Porque tenemos varias cámaras. Esto también lo solucionaremos en el siguiente vídeo.

9. Propiedad (owner)

Solo el propietario puede instanciar y mover esa instancia, así como sincronizarse con el resto. Debemos modificar la script, para que si soy el propietario pueda moverlo.

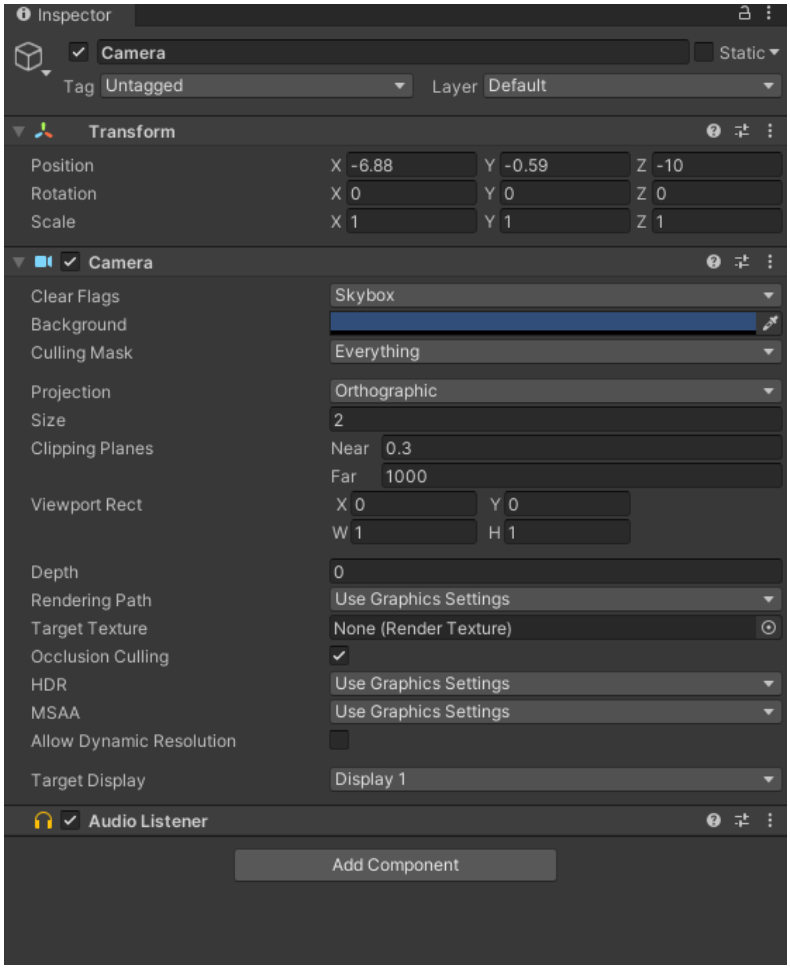
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

using Photon.Pun;

Script de Unity (2 referencias de recurso) | 0 referencias
public class Player : MonoBehaviour
{
    public float speed, jumpForce;
    private Rigidbody2D rig;
    private Animator anim;


    // Start is called before the first frame update
    Mensaje de Unity | 0 referencias
    void Start()
    {
        if (GetComponent <PhotonView>().IsMine) //Si no es mío este script
        {
            rig = GetComponent<Rigidbody2D>();
            anim = GetComponent<Animator>();
        }

    }
```

```
// Update is called once per frame
Mensaje de Unity | 0 referencias
void Update()
{// Le damos velocidad pero también en el eje y porque sino se quedaría parado.

    if (GetComponent<PhotonView>().IsMine)
    {
        rig.velocity = (transform.right * speed * Input.GetAxis("Horizontal")) + (transform.up * rig.velocity.y);

        if (rig.velocity.x > 0.1f) //Cambiamos la imagen de movimiento
            GetComponent<SpriteRenderer>().flipX = false;
        else if (rig.velocity.x < 0.1f)
            GetComponent<SpriteRenderer>().flipX = true;

        //añadimos el salto
        if (Input.GetButtonDown("Jump"))
        {
            rig.AddForce(transform.up * jumpForce);
        }
        //Añadimos la animación.
        anim.SetFloat("velocityX", Mathf.Abs(rig.velocity.x));
        anim.SetFloat("velocityY", rig.velocity.y);
    }
```

Ahora podemos comprobar que cada jugador mueve su jugador. Aunque no sincroniza el flip y la cámara de cada jugador, pues estamos instanciando dos cámaras.

10. Sincronización Cámara

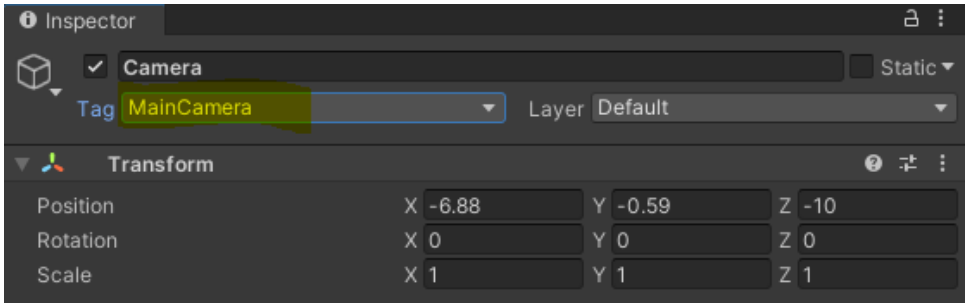
Para sincronizar la cámara, lo primero nos tenemos que crear una cámara. La cámara con la siguiente configuración:



A los dos prefabs le quitamos su cámara. Cuando se instancie cada jugador le ponga la cámara:

```
Mensaje de Unity | 0 referencias
void Start()
{
    if (GetComponent <PhotonView>().IsMine) //Si no es mío este script
    {
        rig = GetComponent<Rigidbody2D>();
        anim = GetComponent<Animator>();
        //Ojo las cámaras no se sincronizan
        Camera.main.transform.SetParent(transform);
        Camera.main.transform.position = transform.position + (Vector3.up) +transform.forward*-10);
    }

}
```

Deberemos decirle que la cámara de la escena es principal, eso se hace dándole un tag como main camera:



11. RPC flip usuario

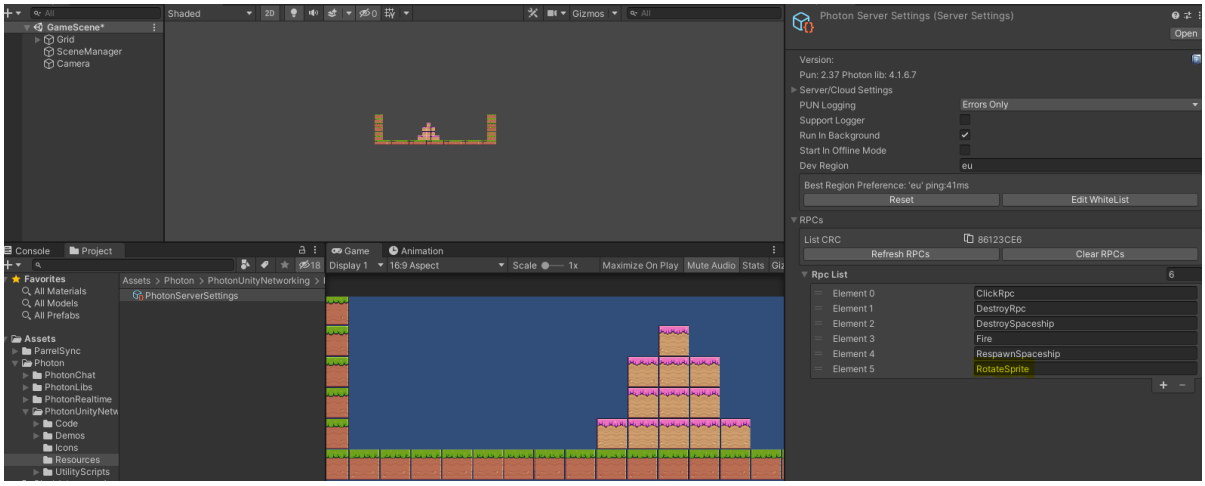
Nos falta sincronizar la rotación o flip del usuario. Para eso tenemos que llamar a una función rpc. Es decir, que no se pueden sincronizar con Photon Network.

Vamos a sincronizar con Photon View. Para eso utilizaremos una llamada para que se ejecuten en todos los usuarios:

```
void Update()
{// Le damos velocidad pero también en el eje y porque sino se quedaría parado.

    if (GetComponent<PhotonView>().IsMine)
    {
        rig.velocity = (transform.right * speed * Input.GetAxis("Horizontal")) + (transform.up * rig.velocity.y);

        if (rig.velocity.x > 0.1f) //Cambiamos la imagen de movimiento
            GetComponent<PhotonView>().RPC("RotateSprite", RpcTarget.All, false);
        else if (rig.velocity.x < 0.1f)
            GetComponent<PhotonView>().RPC("RotateSprite", RpcTarget.All, true);


        //añadimos el salto
        if (Input.GetButtonDown("Jump"))
        {
            rig.AddForce(transform.up * jumpForce);
        }
        //Añadimos la animación.
        anim.SetFloat("velocityX", Mathf.Abs(rig.velocity.x));
        anim.SetFloat("velocityY", rig.velocity.y);
    }
}
```

```
[PunRPC]
0 referencias
public void RotateSprite(bool rotate)
{
    GetComponent<SpriteRenderer>().flipX = rotate;


}
```

Lo podemos ver como se ha insertado:



Para que no se ejecute en cada Frame:

```
if (rig.velocity.x > 0.1f && GetComponent<SpriteRenderer>().flipX) //Cambiamos la imagen de movimiento
    GetComponent<PhotonView>().RPC("RotateSprite", RpcTarget.All, false);
else if (rig.velocity.x < 0.1f && GetComponent<SpriteRenderer>().flipX)
    GetComponent<PhotonView>().RPC("RotateSprite", RpcTarget.All, true);
```